

Laboratorio di Informatica 1

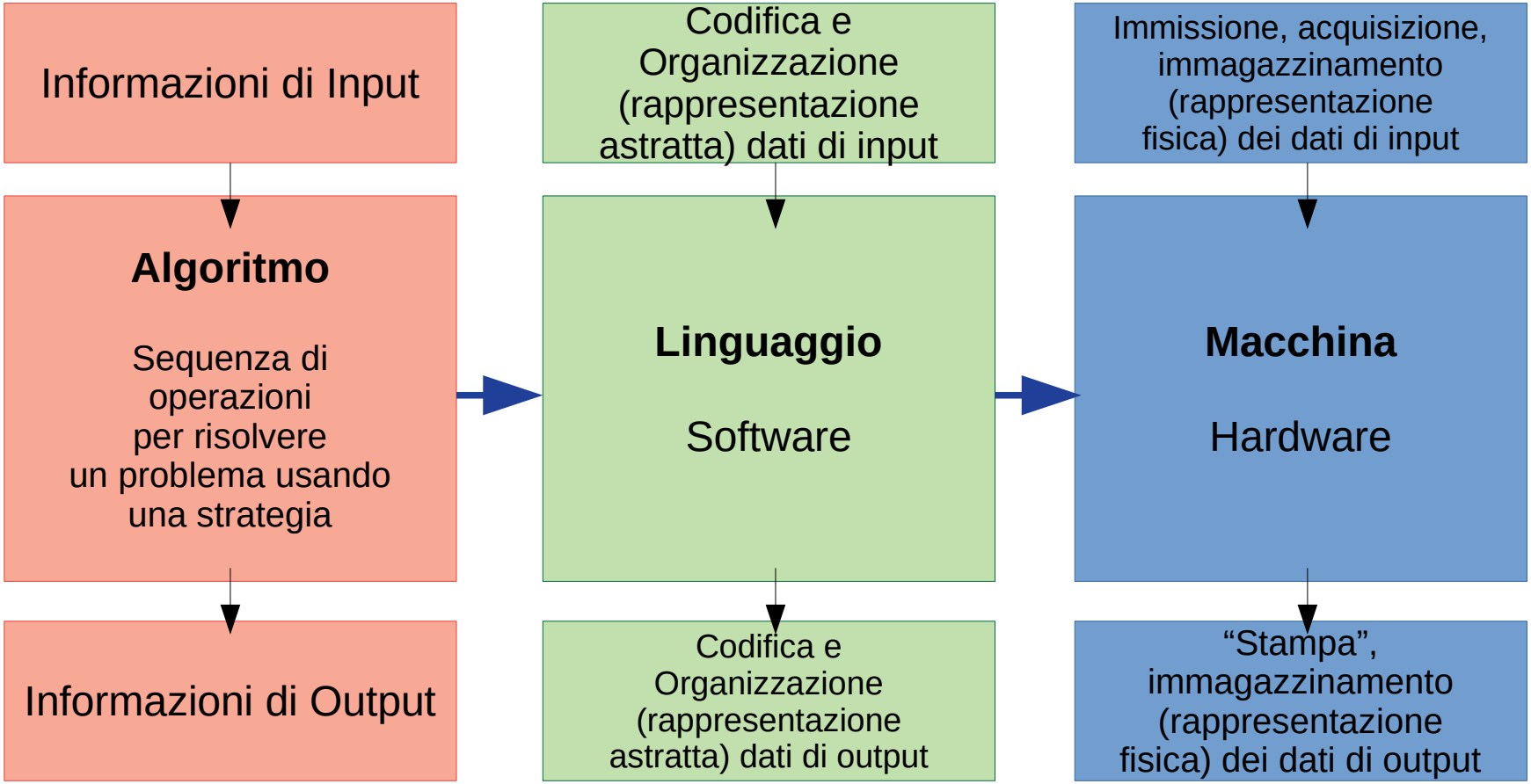
Corso di Laurea in Fisica

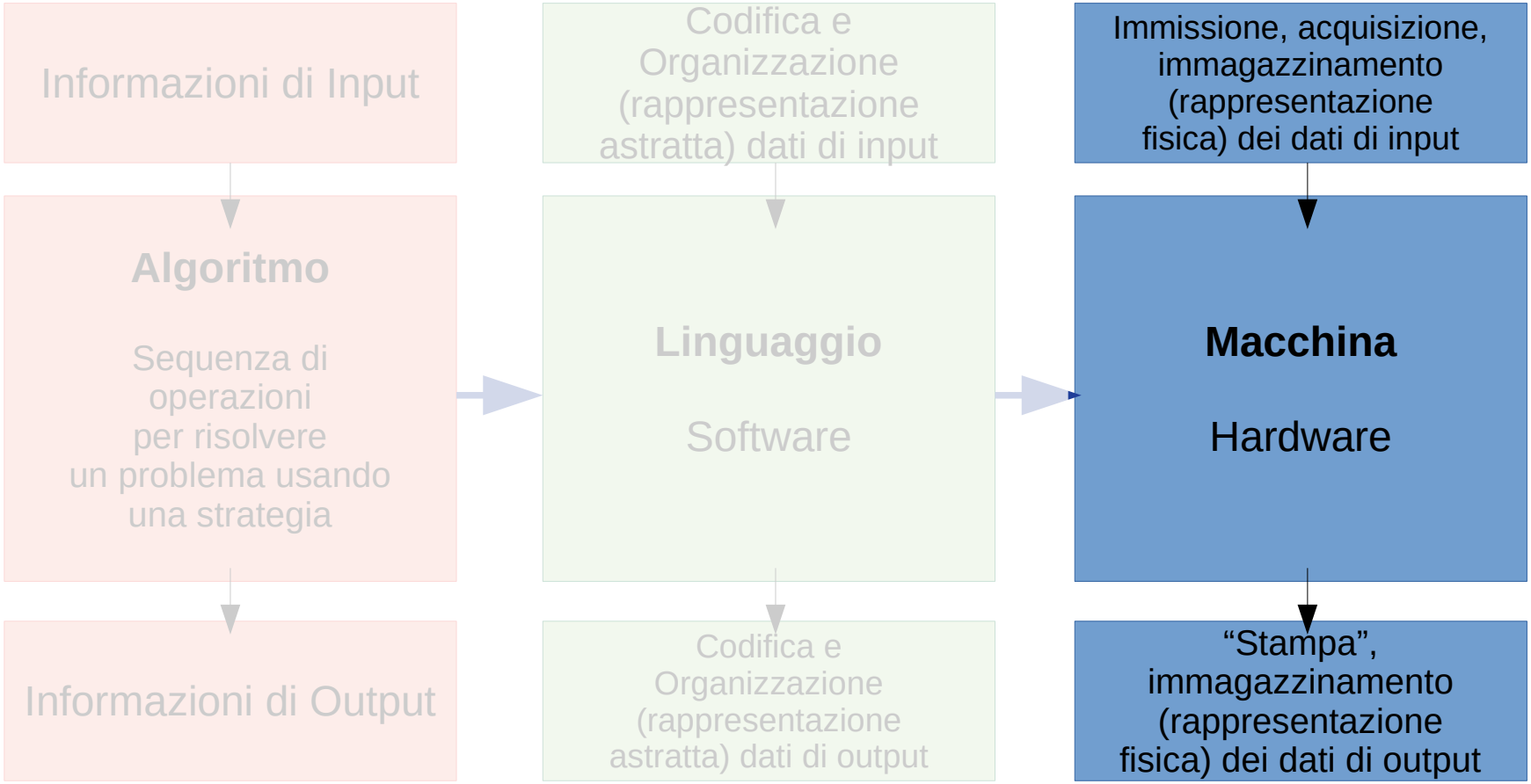
Federico Nati, Uni. Milano-Bicocca, 2021

Lezione 2, 10 marzo 2021

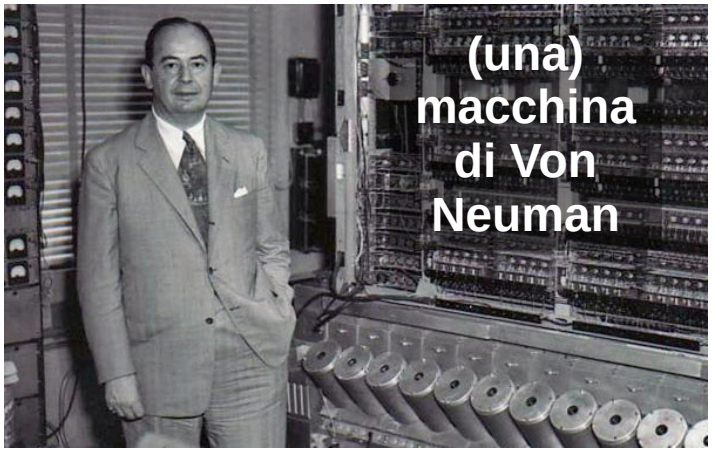
La volta scorsa abbiamo visto:

- Cosa è l'informatica
- Cosa sono gli algoritmi
- Cosa significa rappresentare informazioni in modo digitale
- Bit, Byte, Base 2 e Base 16
- Cosa sono i linguaggi di programmazione
- Cosa sono le strutture di controllo
- Primi esempi di codice


Informazioni di Input
Codifica e Organizzazione (rappresentazione astratta) dati di input
Immissione, acquisizione, immagazzinamento (rappresentazione fisica) dei dati di input
Algoritmo
Sequenza di operazioni per risolvere un problema usando una strategia
Linguaggio
Software
Macchina
Hardware
Informazioni di Output
Codifica e Organizzazione (rappresentazione astratta) dati di output
“Stampa”, immagazzinamento (rappresentazione fisica) dei dati di output


Informazioni di Input
Codifica e Organizzazione (rappresentazione astratta) dati di input
Immissione, acquisizione, immagazzinamento (rappresentazione fisica) dei dati di input
Algoritmo
Sequenza di operazioni per risolvere un problema usando una strategia
Linguaggio
Software
Macchina
Hardware
Informazioni di Output
Codifica e Organizzazione (rappresentazione astratta) dati di output
“Stampa”, immagazzinamento (rappresentazione fisica) dei dati di output

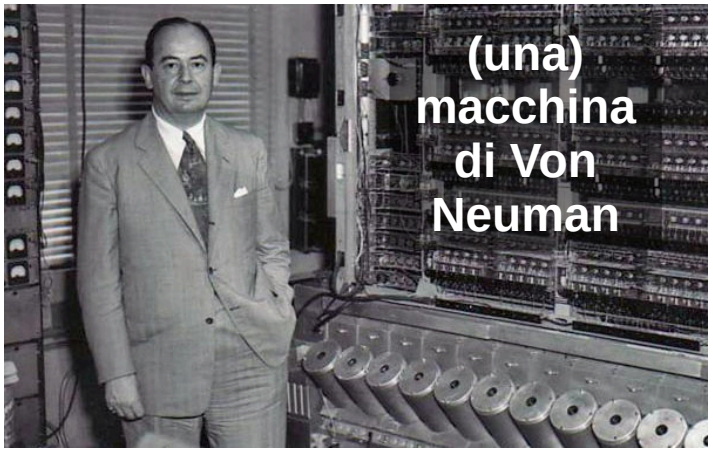
Architettura di Von Neuman



John von Neumann, Budapest, 1903 – Washington, 1957), è stato un matematico, fisico e informatico ungherese naturalizzato statunitense.

In realtà dobbiamo l'architettura che porta il suo nome anche a molte altre persone che hanno contribuito all'idea.

Architettura di Von Neuman



John von Neumann, Budapest, 1903 – Washington, 1957), è stato un matematico, fisico e informatico ungherese naturalizzato statunitense.

In realtà dobbiamo l'architettura che porta il suo nome anche a molte altre persone che hanno contribuito all'idea.

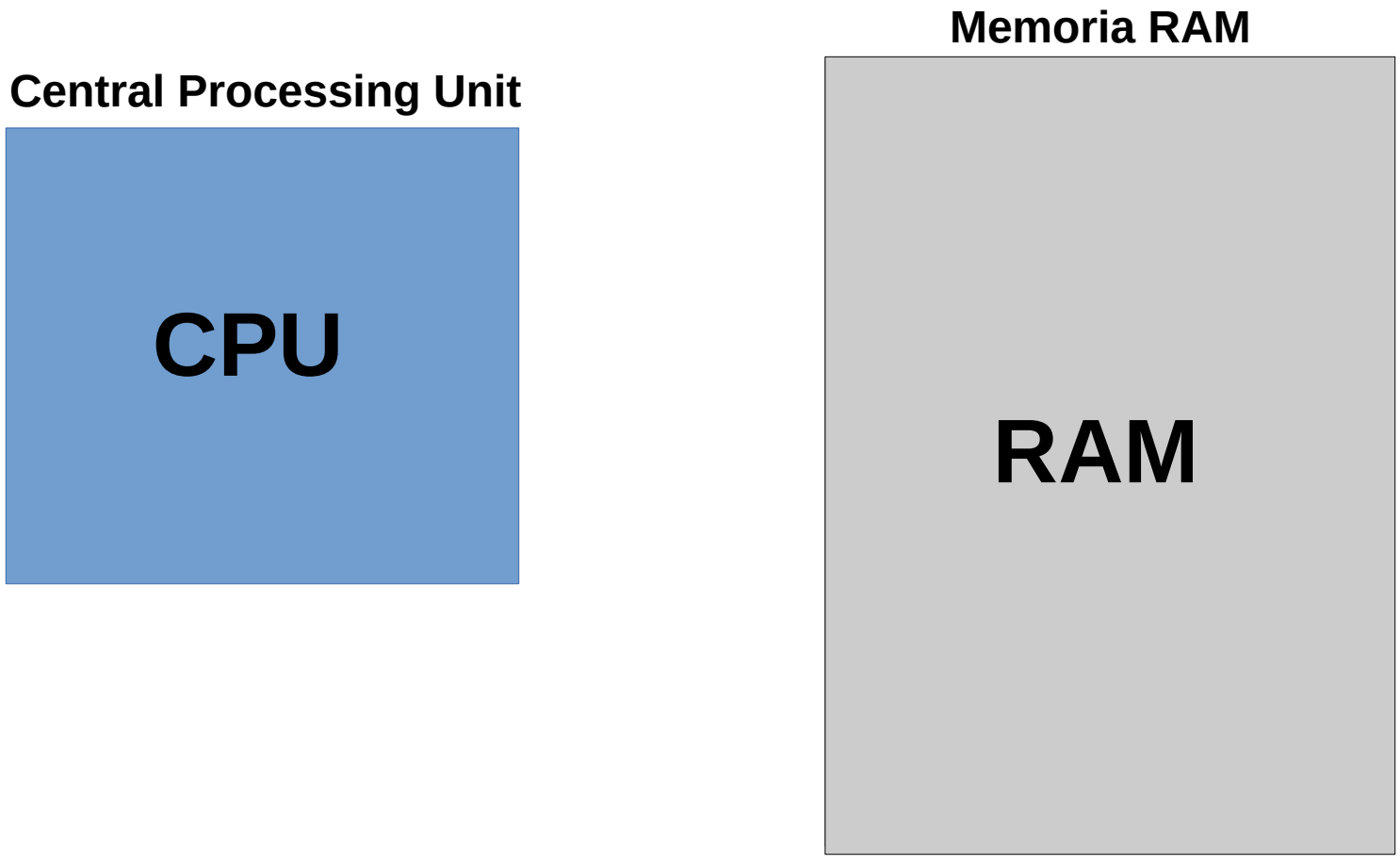
DA NON CONFONDERE CON

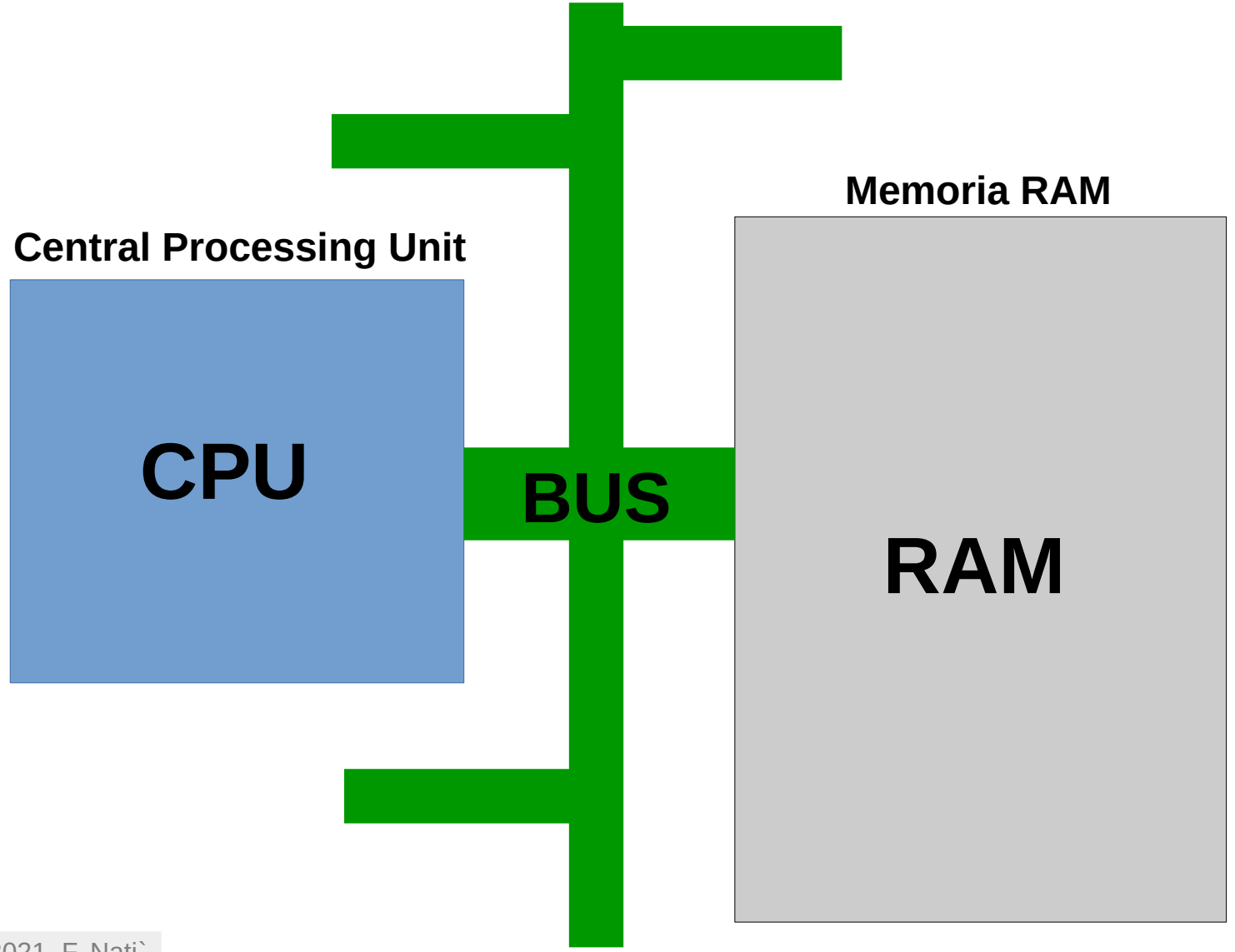


(una) macchina di Paul Neuman

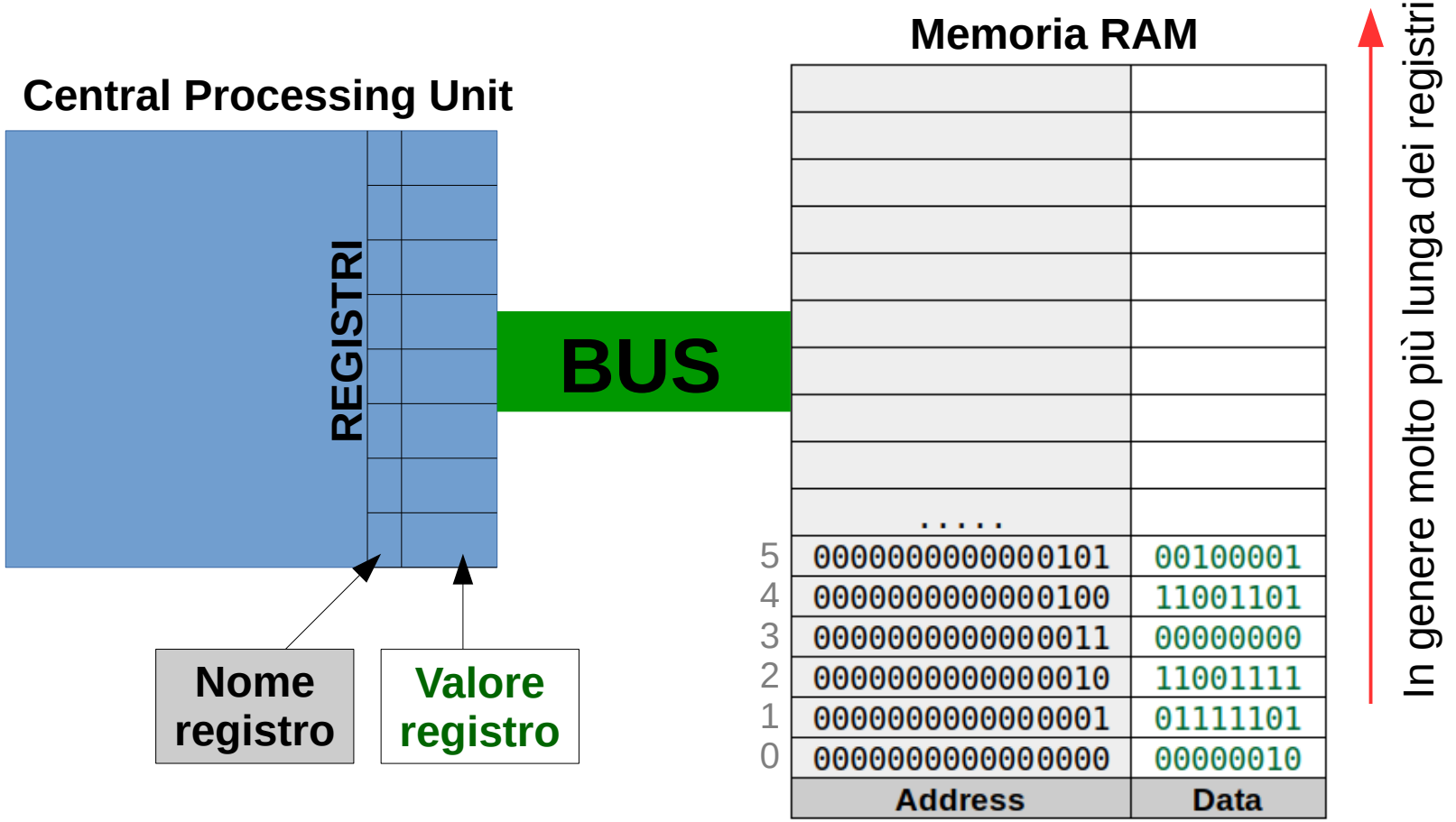
Paul Newman (1925 - 2008) è stato uno dei più famosi attori, registi, produttori cinematografici statunitensi.

Architettura di Von Neumann

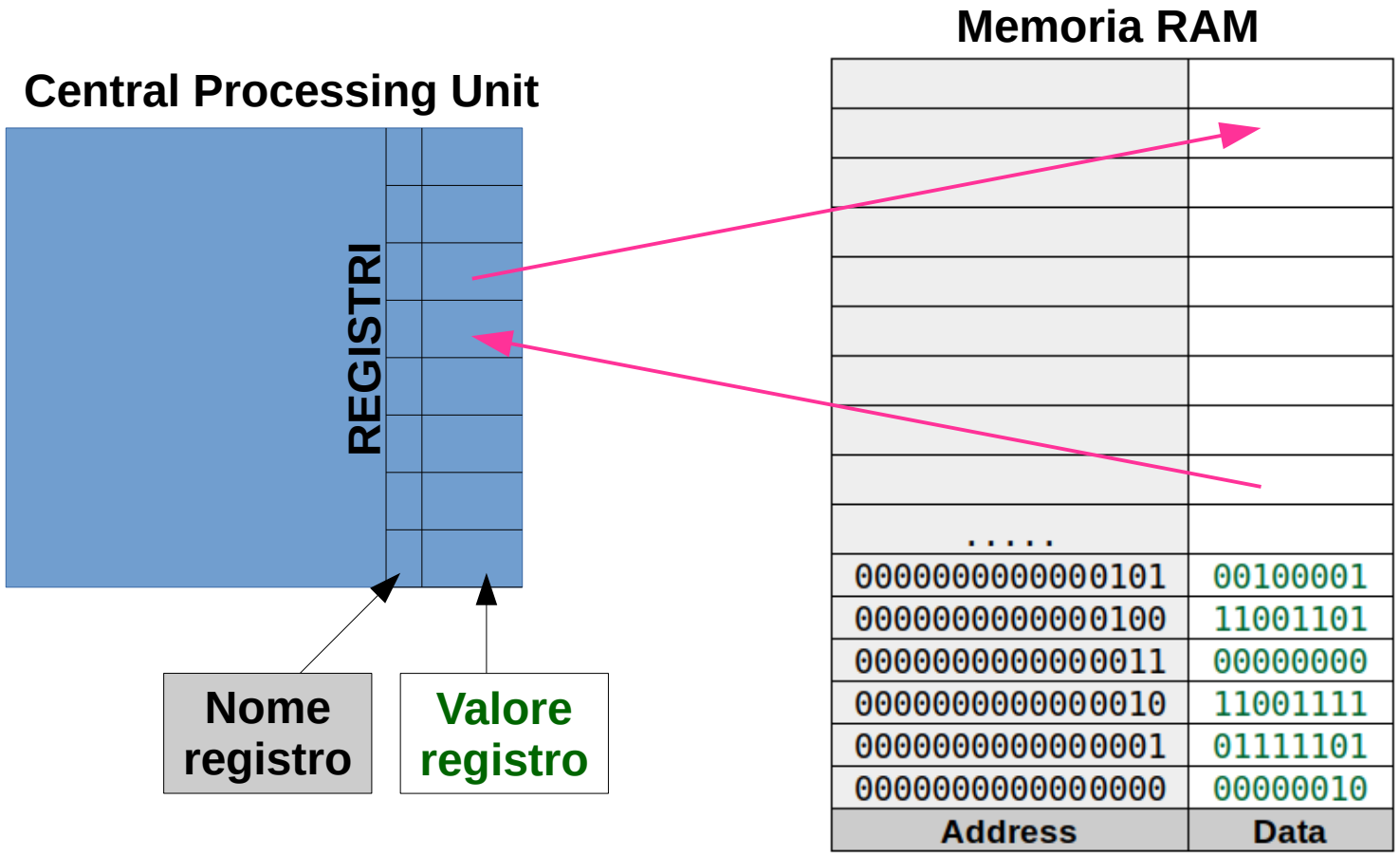


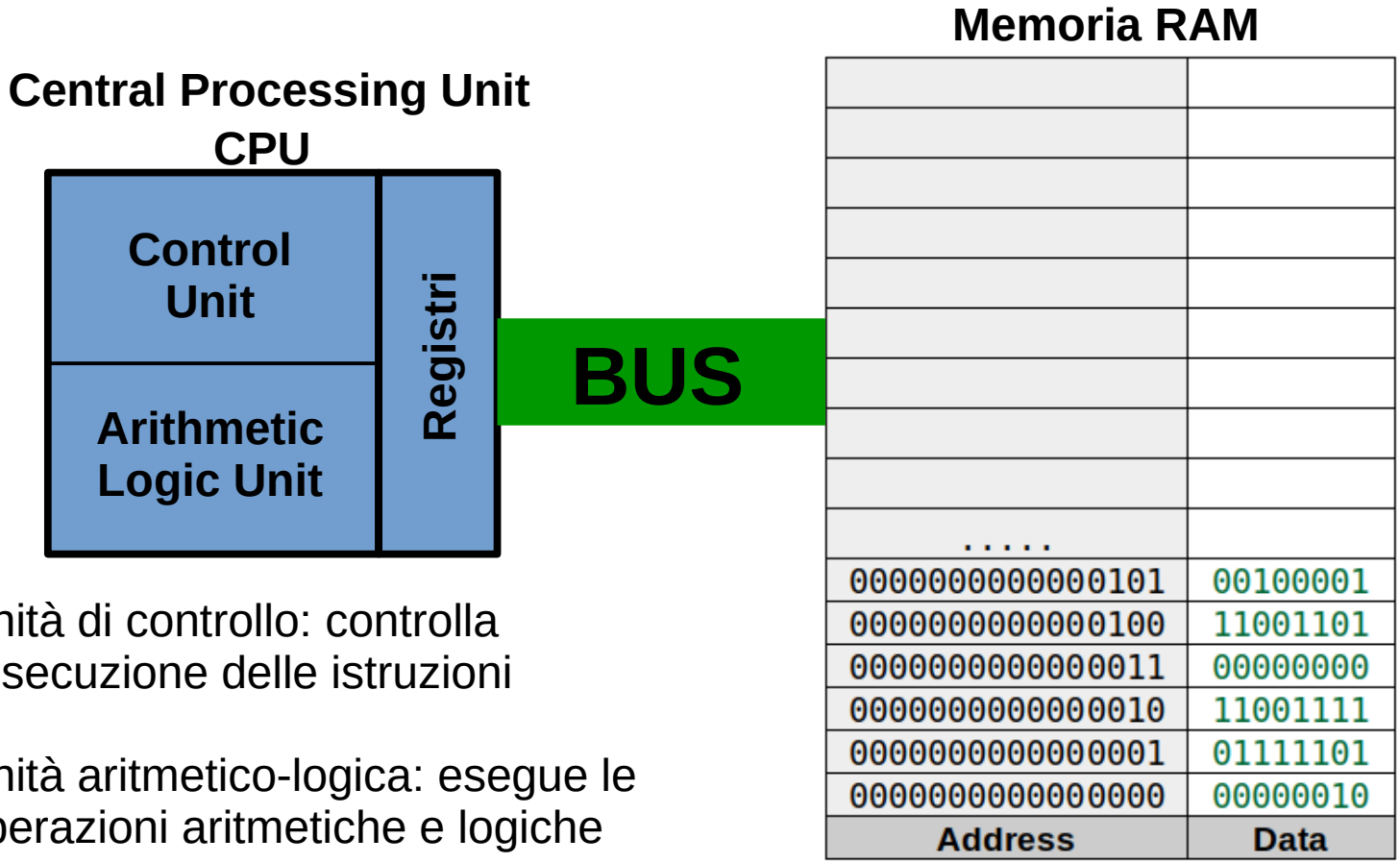

Memoria RAM
Central Processing Unit
CPU
BUS
RAM

Architettura di Von Neumann



Usando gli indirizzi si può accedere alle celle di memoria in modo non sequenziale. Ecco perché si chiama RANDOM ACCESS MEMORY.

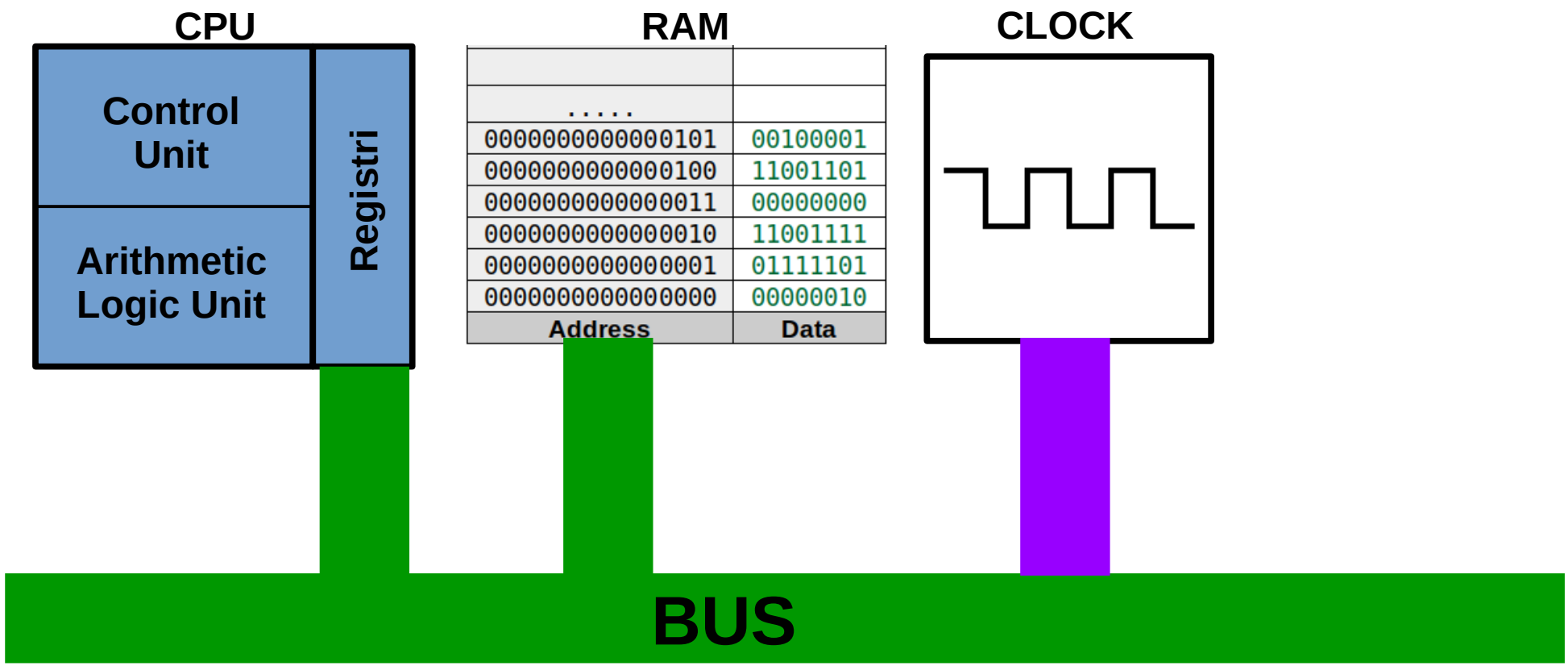




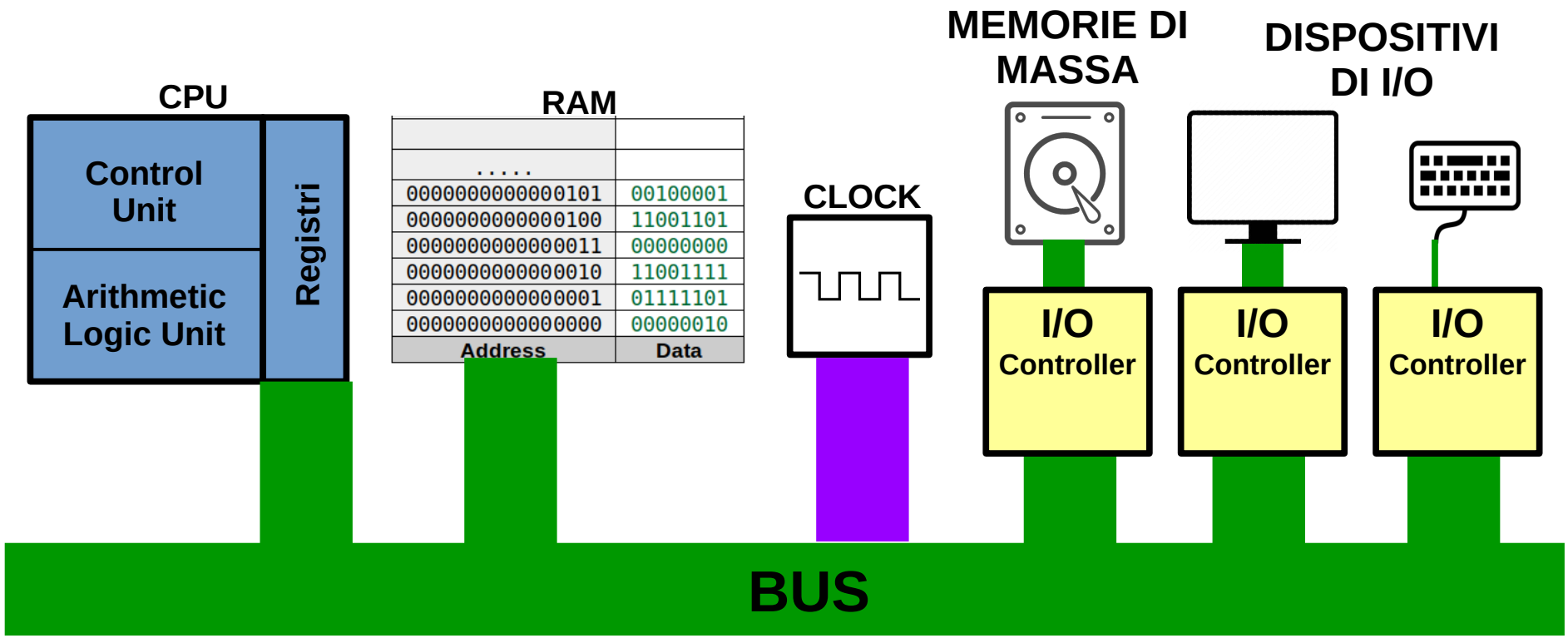
Unità di controllo: controlla l'esecuzione delle istruzioni

Unità aritmetico-logica: esegue le operazioni aritmetiche e logiche

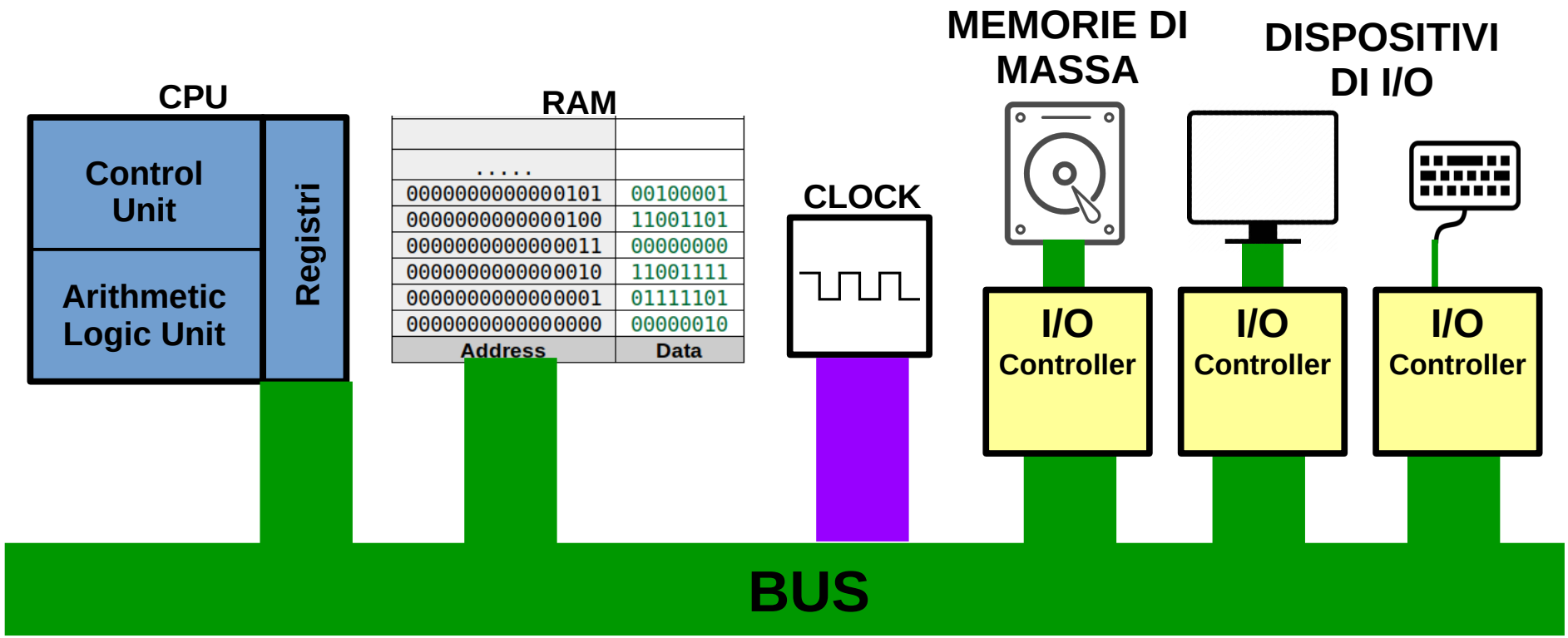
Architettura di Von Neumann



Architettura di Von Neumann



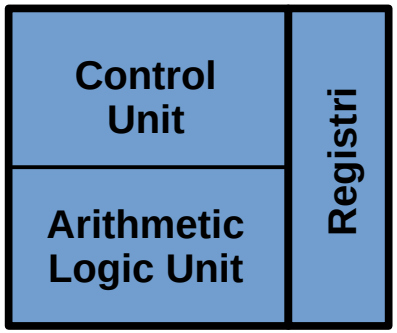
Architettura di Von Neumann



Essa prevede anche che i programmi in esecuzione e i dati con i quali operano vengano caricati nella RAM, cioè utilizzino la stessa memoria.

Ci sono memorie e memorie...

CPU



- I registri sono la memoria più veloce che ha a disposizione la CPU
- Ma sono dell'ordine delle decine e contengono 2, 4, 8 byte
- La dimensione di un registro limita il numero di indirizzi della memoria RAM rappresentabili
- Volatile

RAM

.....	
0000000000000101	00100001
0000000000000100	11001101
0000000000000011	00000000
0000000000000010	11001111
0000000000000001	01111101
0000000000000000	00000010
Address	**Data**

- Molto più veloce delle unità fisse
- Più lenta dei registri
- Volatile
- Molto più grande dei registri
- Molto più piccola delle unità fisse
- Oggi è dell'ordine dei GigaByte (10^9 byte)

Memorie di massa



- Non volatile
- Molto più grande della RAM
- Molto più lenta della RAM
- Oggi è dell'ordine dei TeraByte (10^{12} byte)

Volatile = Perde le informazioni contenute se si spegne il computer

Nota terminologica

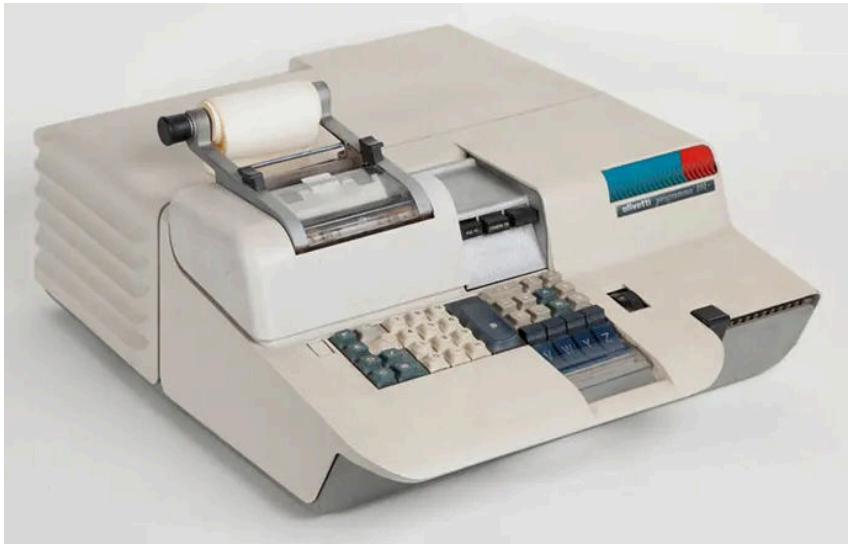
I termini tra parentesi quadre sono spesso usati come sinonimi:

- **[CPU, microprocessore, unità centrale, processore]**
- **[Memoria, memoria di lavoro, RAM]**
- **[Disco, memoria di massa, drive, disco fisso, disco rigido, hard disk, hard drive]**
  - In effetti *hard disk* si riferisce a una specifica classe, oggi si distinguono dischi rigidi e dischi a stato solido, per i quali si usa: SSD, disco a stato solido, memoria a stato solido
- A volte *chip* o *microchip*, che è un termine più generico, viene usato per indicare il processore, o “un” processore.A volte chip o microchip, che è un termine più generico, viene usato per indicare il processore, o “un” processore.
- Memoria *chace*, o *cache*, è un generico termine che si riferisce a uno spazio che immagazzina informazioni ad accesso rapido senza che si debba interpellare la memoria principale
- Il termine “memoria” usato da solo normalmente si riferisce alla RAM
- I dischi “esterni” sono sempre memorie di massa, e sono per esempio le chiavette USB, i dischi (o drive) esterni USB, i CD/DVD, floppy disk o i nastri magnetici
- Il termine *drive* indica il dispositivo che gestisce un generico supporto di massa

Hardware

- Veniamo al cosiddetto hardware, che in inglese significa strumenti, attrezzi, ferramenta, materiale
- I computer sono di diverse forme, dimensioni, velocità, versatilità, e sono naturalmente evoluti dai primi esemplari a oggi moltissimo
- Tuttavia da un punto di vista funzionale i concetti chiave sono quelli espressi per l'architettura di Von Neumann, dalla quale la grande maggioranza non si discosta o si discosta poco
- Vediamo come appaiono questi “strumenti”, cioè l'hardware

Il primo Personal Computer



Olivetti 101, 1965, primo computer da scrivania della storia, 3200$.

La NASA ne acquistò decine di esemplari per elaborare la traiettoria del viaggio della missione Apollo 11, che nel 1969 portò l'uomo sulla luna.

La rivoluzione stava nella dimensione ridotta, nel costo relativamente contenuto, nella possibilità di risolvere in poco tempo calcoli complessi, generati e controllati da programmi scritti attraverso una tastiera, usando un linguaggio che comprendeva sedici istruzioni diverse.

Integrava una piccola stampante, un lettore di schede perforate per "caricare" i programmi precedentemente scritti, e una memoria magnetica per conservare i dati.

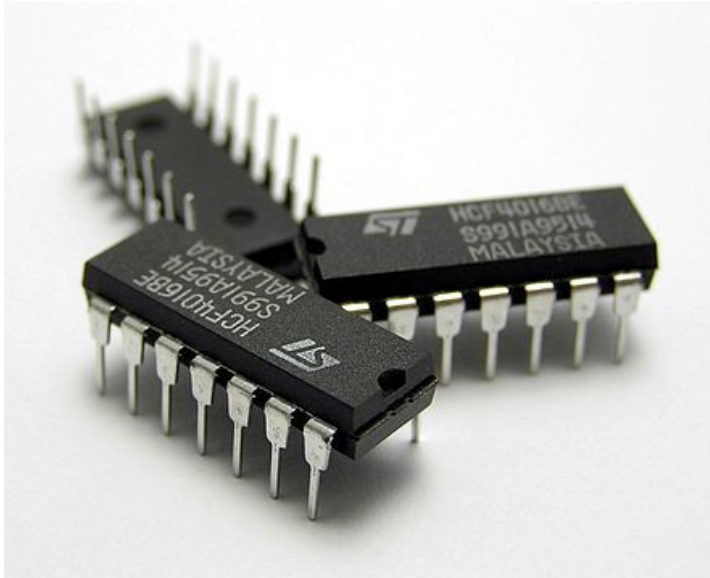
Circuiti elettronici integrati (chip)

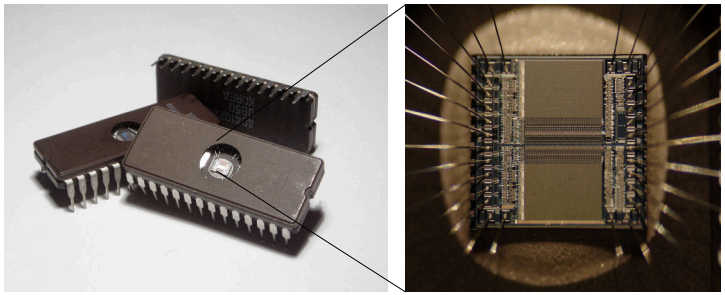
Un circuito integrato è un circuito elettronico miniaturizzato dove i componenti, in particolare i transistor, sono costruiti grazie a un unico processo fisico-chimico.

Il supporto che contiene gli elementi (attivi o passivi) che costituiscono il circuito è una piastrina miniaturizzata di silicio che viene chiamata *chip*; spesso si utilizza il termine chip per indicare complessivamente l'integrato.

Il circuito integrato è adibito, sotto forma di rete logica digitale o analogica, a funzionalità di processamento o elaborazione di ingressi espressi sotto forma di segnali elettrici, al fine di ottenere dati in uscita.

L'ideazione del circuito integrato si deve a Jack St. Clair Kilby, che nel 1958 ne costruì il primo esemplare composto da circa dieci componenti elementari, per il quale vinse il premio Nobel per la Fisica nel 2000.





Dentro un calcolatore

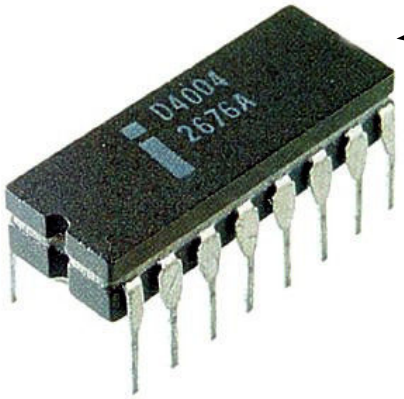




Il microprocessore

Il microprocessore è un insieme di circuiti elettronici integrati, quindi di dimensioni molto ridotte. Esso viene utilizzato per la realizzazione delle CPU impiegate dalla quasi totalità dei moderni computer, con la caratteristica di utilizzare, per tutte le sue elaborazioni, un insieme di istruzioni fondamentali di base.

Federico Faggin (Vicenza, 1941), fisico e imprenditore italiano naturalizzato statunitense, fu il principale responsabile della realizzazione del primo microprocessori monolitico, l'Intel 4004 (1971).



Anni '80, processore Intel a 16 bit, 1 core a ~10 MHz



Anni 2020, processore AMD a 64 bit, 12 core a ~3 GHz



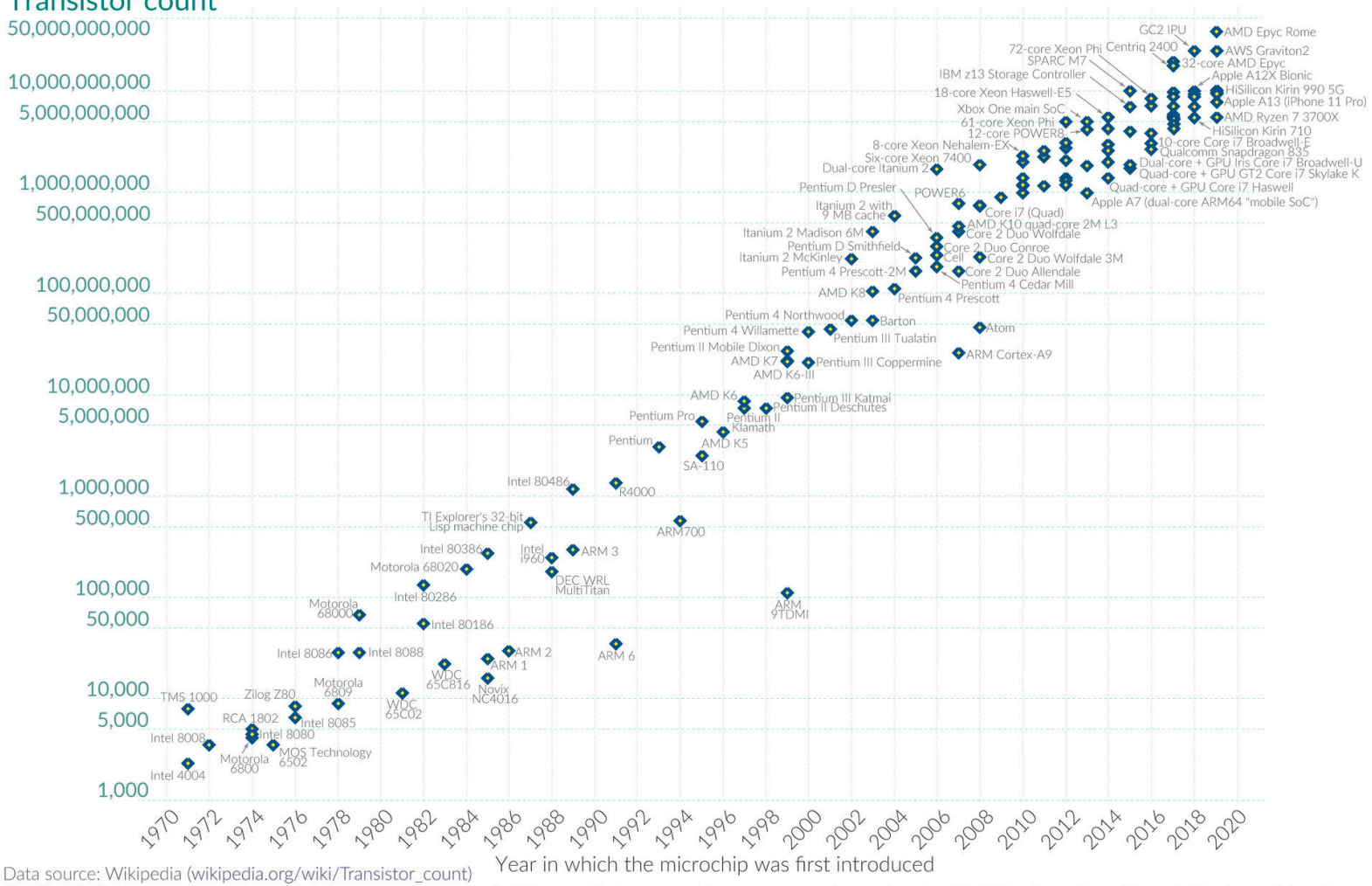
Miniaturizzare = Velocizzare

- La velocita di clock determina il numero di istruzioni al secondo. Ogni istruzione consuma (e dissipa sotto forma di calore) energia
- Transistor più piccoli hanno una capacità di *gate* inferiore
- Transistor più piccoli si scaldano meno, perché richiedono meno corrente, quindi meno energia per istruzione
- Le distanze sono inferiori, quindi i campi elettrici si propagano tra le connessioni in un tempo inferiore
- Ad alta frequenza le dimensioni dei circuiti contano, perché la lunghezza d'onda ~ dimensioni del circuito
- Si veda anche facoltativamente: “The Fundamental Phsycial Limits of Computation”, Bennett & Landauer, 2011, collegamento sul sito del corso: https://www.scientificamerican.com/article/the-fundamental-physical-limits-of-computation/

Moore's Law: The number of transistors on microchips doubles every two years

Our World in Data

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years.
This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.



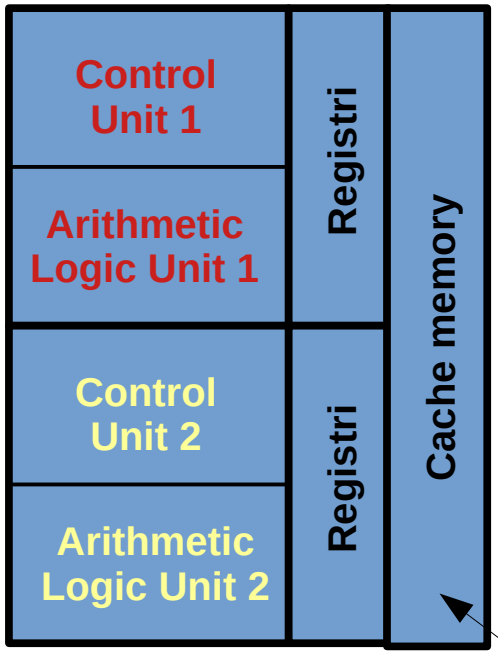
Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)
OurWorldinData.org – Research and data to make progress against the world's largest problems.

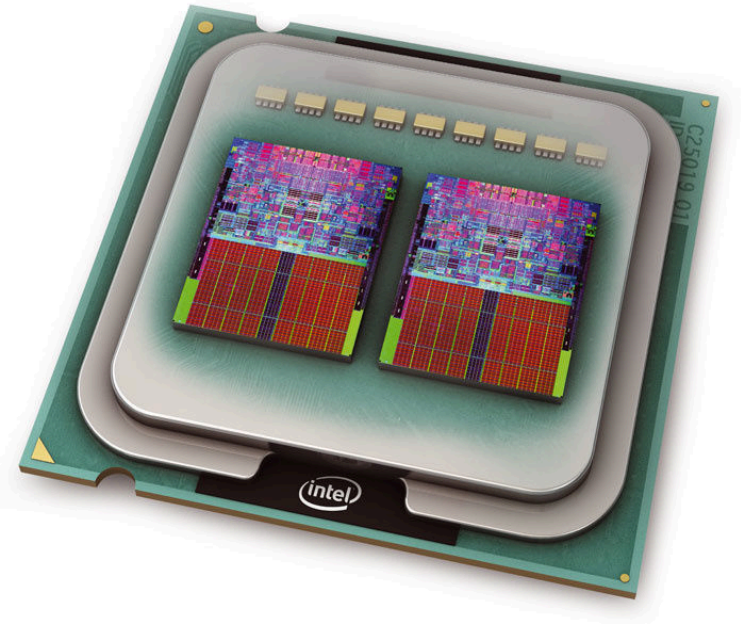
Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

Nota: oggi sono diffuse CPU parallele, multi-core

Oggi i microprocessori in commercio sono **multi-core (es. 2, 4, 6, 8 core)**

La velocità di clock non è il solo parametro per valutare la velocità della macchina



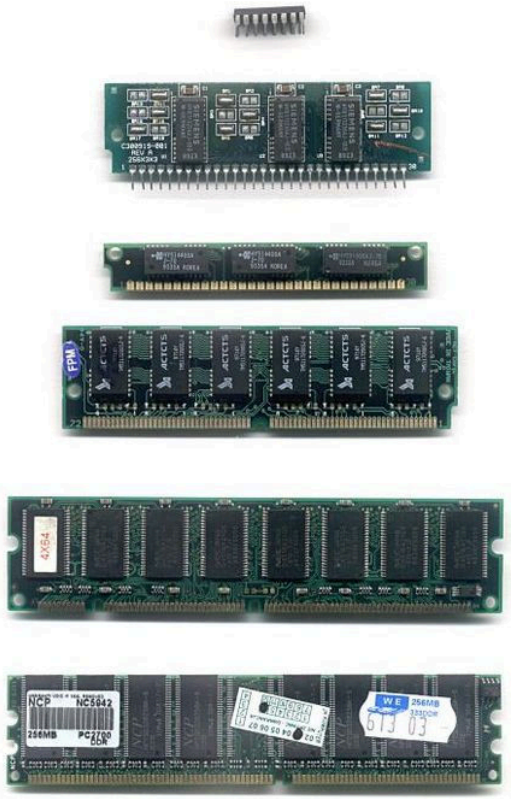


C'è un ulteriore tipo di memoria detto “cache memory”

La RAM

RAM (Random Access Memory, ovvero memoria ad accesso casuale in contrapposizione con la memoria ad accesso sequenziale) è un tipo di memoria **volatile** caratterizzata dal permettere l'**accesso diretto a qualunque indirizzo di memoria con lo stesso tempo di accesso**.

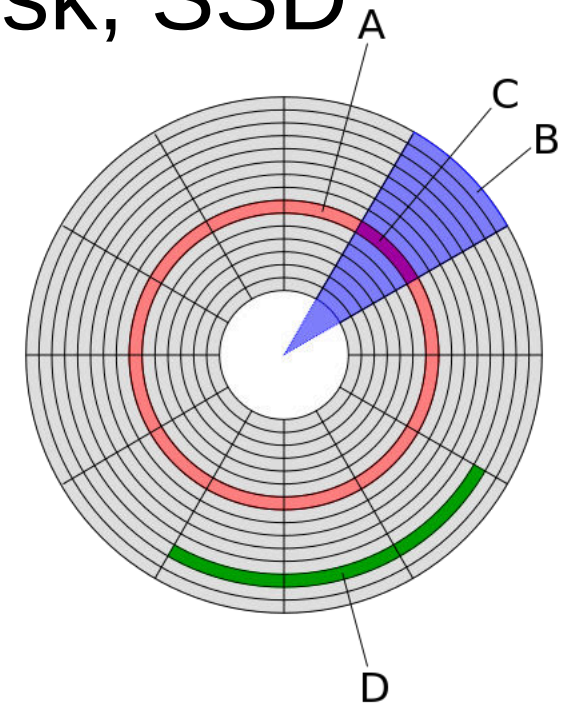
Nella memoria RAM vengono caricati i programmi che la CPU deve eseguire e i dati sui cui si opera, copiandoli da un supporto non volatile.



Disco fisso, Hard Disk, SSD

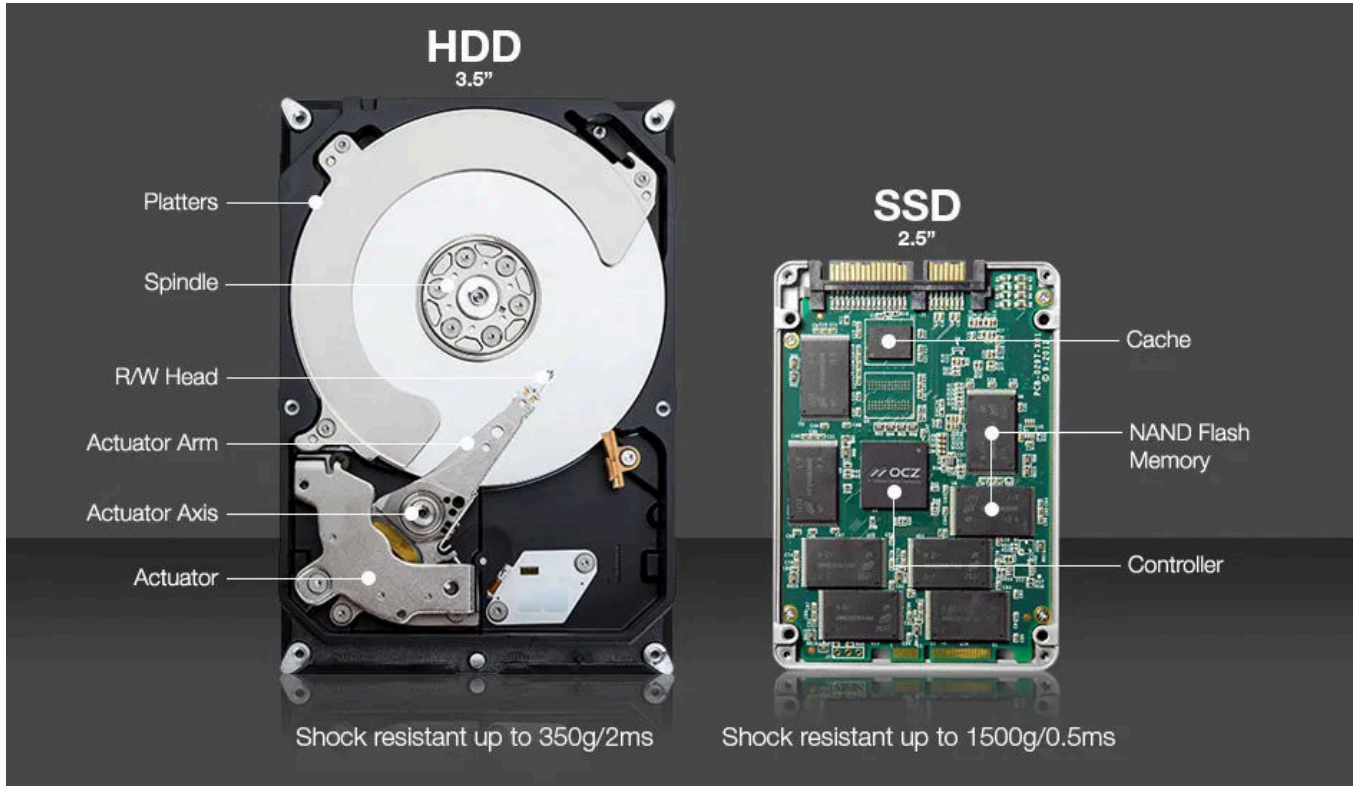






A: Track
B: Geometrical sector
C: Disk sector
D: Cluster

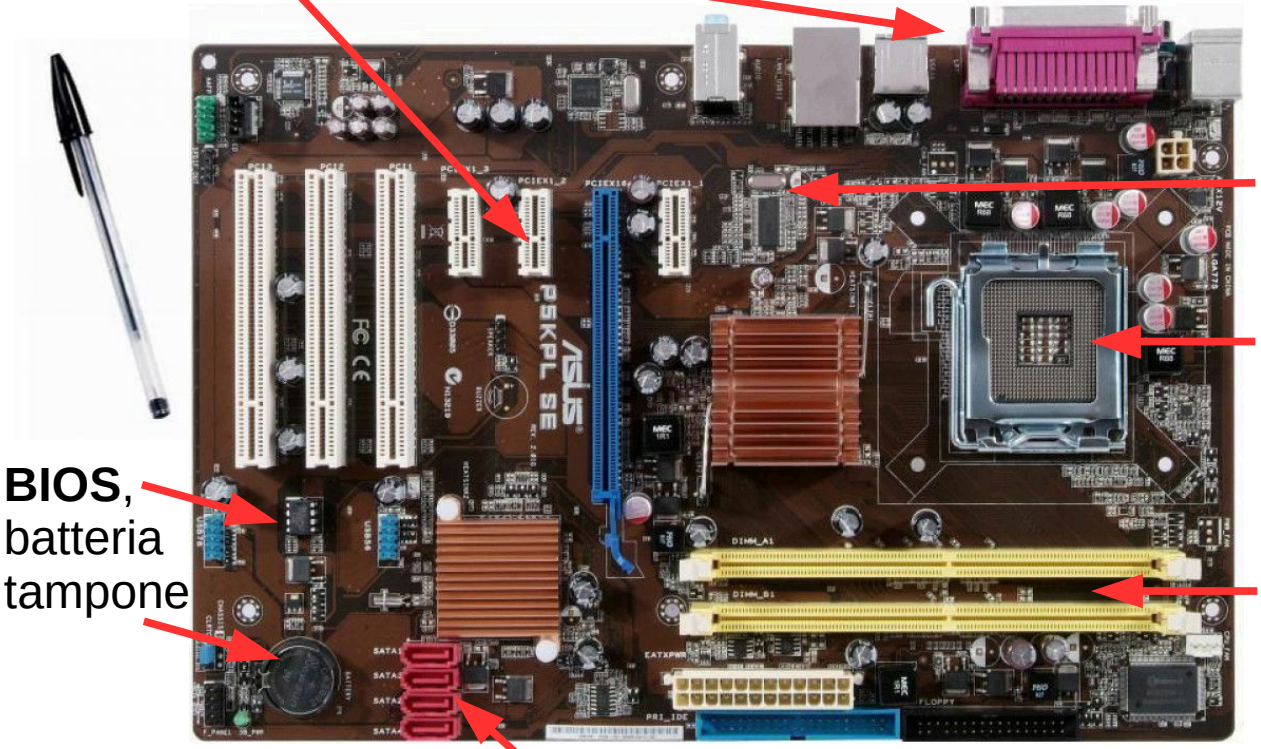
Solid State Drive





Bus e schede madri

I/O (Tastiera, mouse, rete, porte di comunicazione, audio/video, schede di espansione)



CLOCK

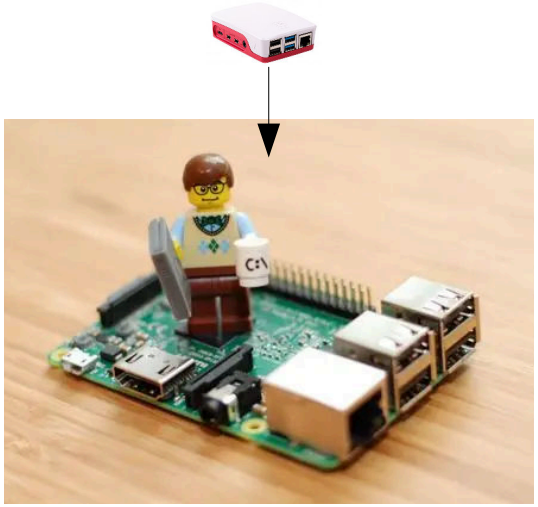
CPU

RAM

BIOS, batteria tampone

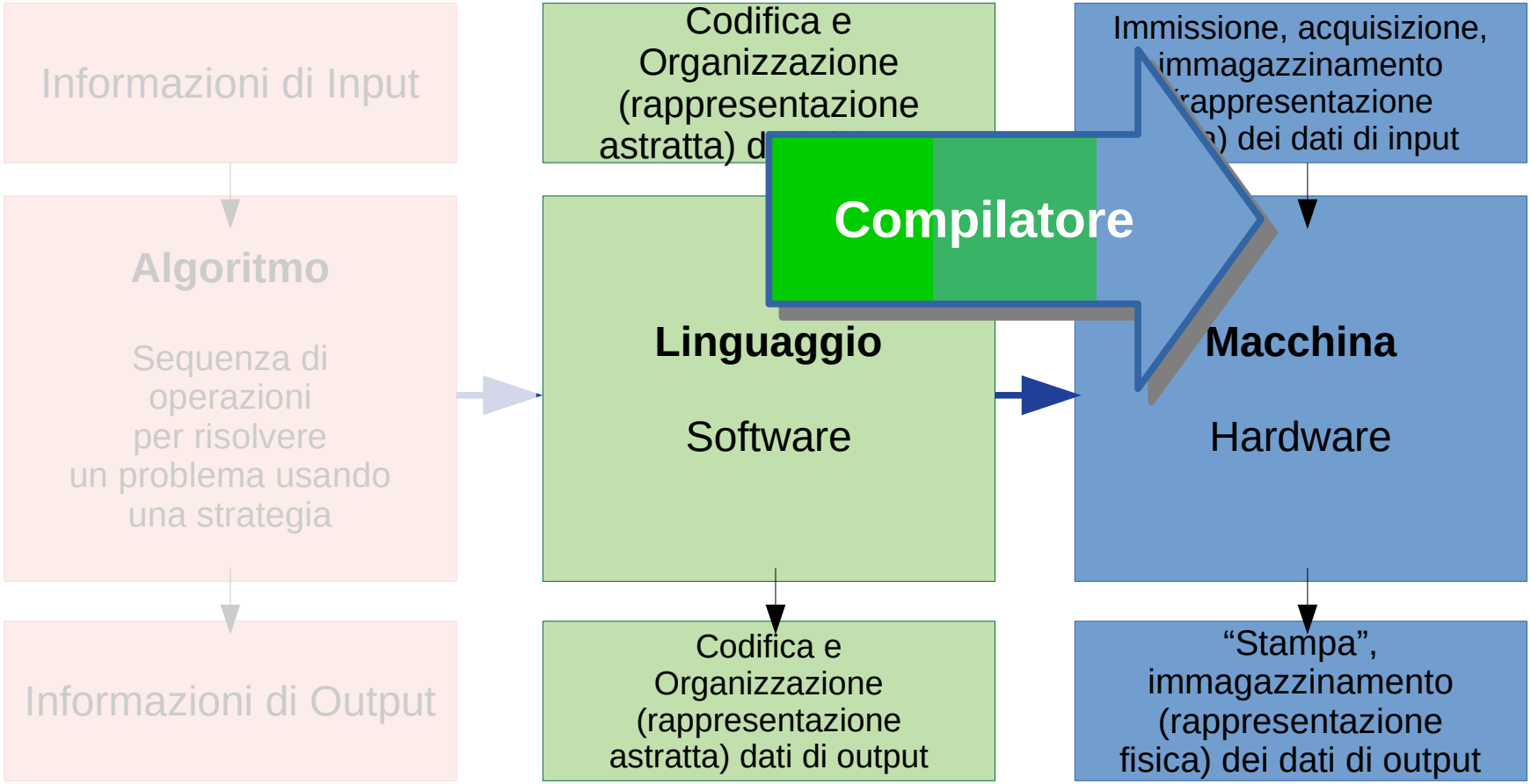
Memorie di massa (HD, SSD)



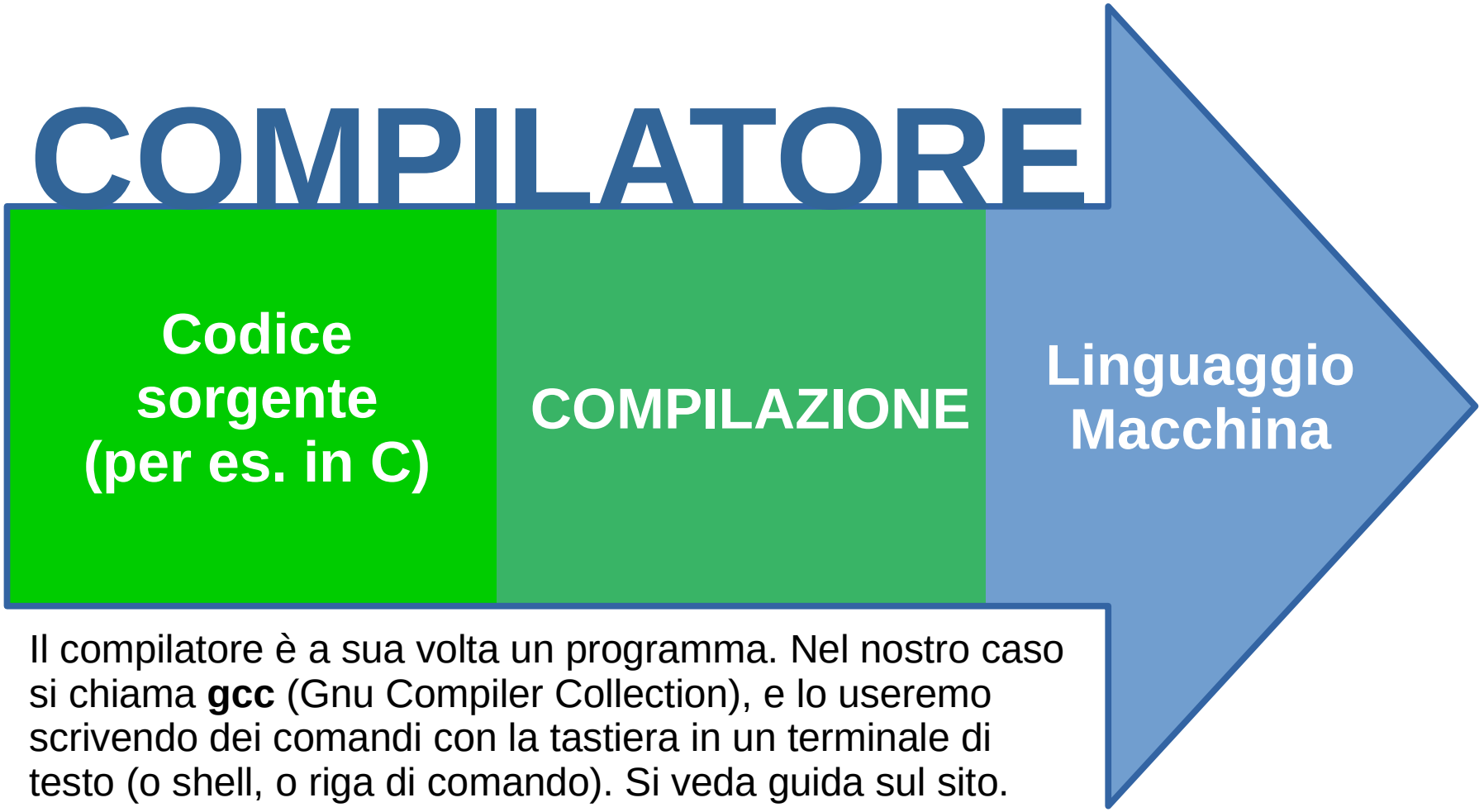


Raspberry PI

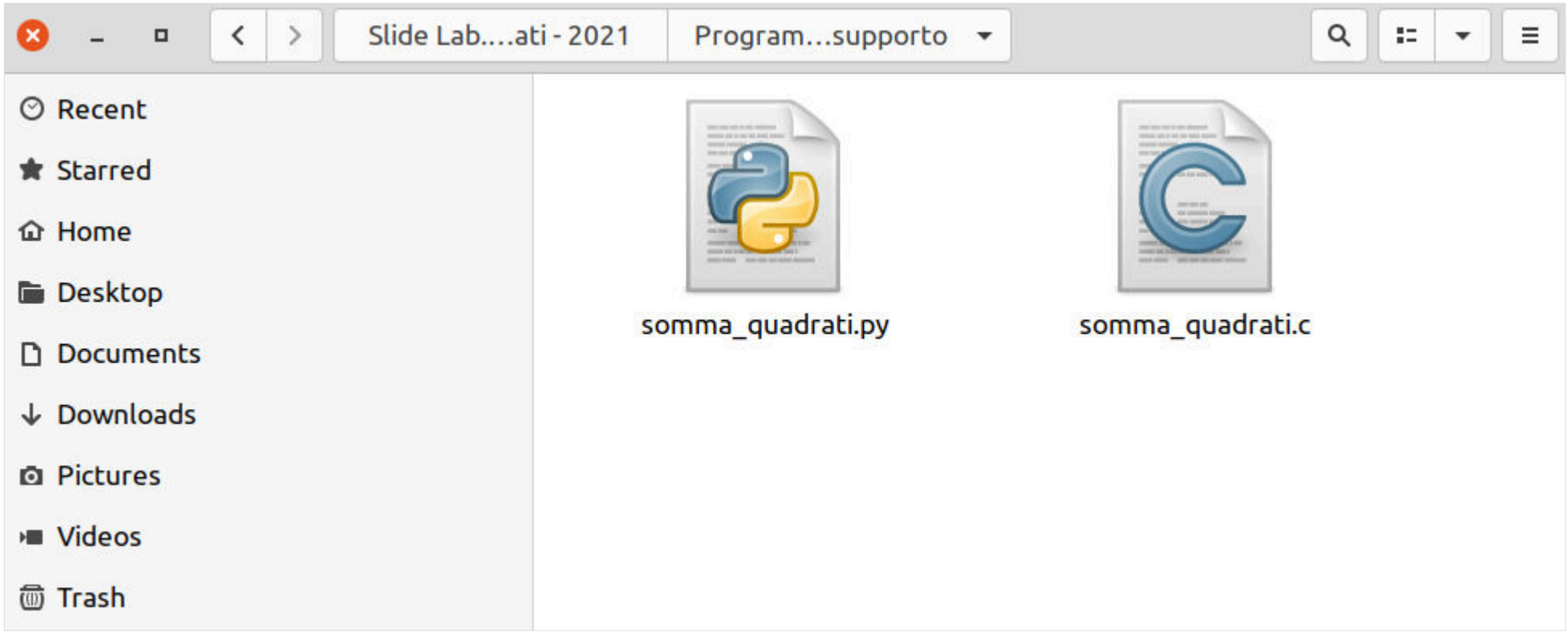
Dal codice sorgente al linguaggio macchina



COMPILATORE



Il compilatore è a sua volta un programma. Nel nostro caso si chiama **gcc** (Gnu Compiler Collection), e lo useremo scrivendo dei comandi con la tastiera in un terminale di testo (o shell, o riga di comando). Si veda guida sul sito.


Slide Lab....ati - 2021
Program...supporto
Recent
Starred
Home
Desktop
Documents
Downloads
Pictures
Videos
Trash
somma_quadrati.py
somma_quadrati.c

Nota terminologica

- Quando si dice “**file**” normalmente si parla di un archivio di dati immagazzinati in una memoria fissa, non volatile.

- In italiano il termine file è ormai nell'uso, ma si possono sentire anche documento, archivio, libreria, eseguibile a seconda dei casi:
  - documento, in genere contiene dati
  - archivio sarebbe la traduzione di file, ma spesso indica un file o una cartella “compressa”, archivio compresso, zip file...
  - libreria si riferisce a software o dati di supporto ad altri programmi

Dal sorgente al programma compilato

somma_quadrati.c

```c
#include <stdio.h>
#include <stdlib.h>

int N = 7;
int i = 0;
int i_quadro;
long int SOMMA = 0;

void main() {

        while (i <= N) {
                i_quadro = i*i;
                SOMMA = SOMMA + i_quadro;
                i += 1;
        }

        printf("%ld\n", SOMMA);

        exit(0);
}
```

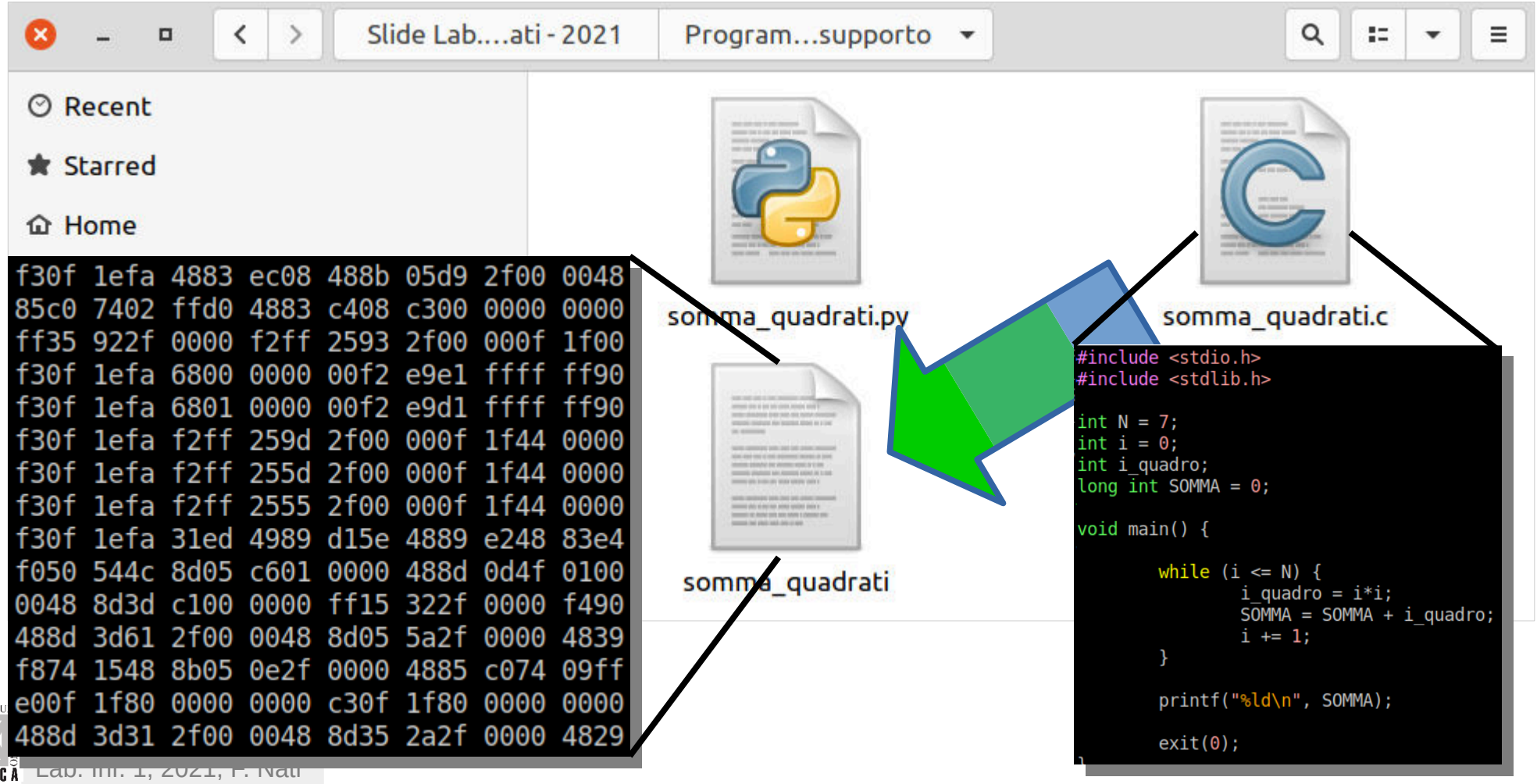
Compilatore

somma_quadrati

```
f30f 1efa 4883 ec08 488b 05d9 2f00 0048
85c0 7402 ffd0 4883 c408 c300 0000 0000
ff35 922f 0000 f2ff 2593 2f00 000f 1f00
f30f 1efa 6800 0000 00f2 e9e1 ffff ff90
f30f 1efa 6801 0000 00f2 e9d1 ffff ff90
f30f 1efa f2ff 259d 2f00 000f 1f44 0000
f30f 1efa f2ff 255d 2f00 000f 1f44 0000
f30f 1efa f2ff 2555 2f00 000f 1f44 0000
f30f 1efa 31ed 4989 d15e 4889 e248 83e4
050 544c 8d05 c601 0000 488d 0d4f 0100
0048 8d3d c100 0000 ff15 322f 0000 f490
488d 3d61 2f00 0048 8d05 5a2f 0000 4839
f874 1548 8b05 0e2f 0000 4885 c074 09ff
e00f 1f80 0000 0000 c30f 1f80 0000 0000
488d 3d31 2f00 0048 8d35 2a2f 0000 4829
```

```
gcc somma_quadrati.c -o somma_quadrati
```

File sorgente e file compilato



Assembly

Codice sorgente in C

```c
#include <stdio.h>
#include <stdlib.h>

int N = 7;
int i = 0;
int i_quadro;
long int SOMMA = 0;

void main() {

        while (i <= N) {
                i_quadro = i*i;
                SOMMA = SOMMA + i_quadro;
                i += 1;
        }

        printf("%ld\n", SOMMA);

        exit(0);
}
```

Assembly

```
SOMMA:
        .zero   8
        .section        .rodata
.LC0:
        .string "%ld\n"
        .text
        .globl  main
        .type   main, @function
main:
.LFB6:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        jmp     .L2
.L3:
        movl    i(%rip), %edx
        movl    i(%rip), %eax
        imull   %edx, %eax
        movl    %eax, i_quadro(%rip)
        movl    i_quadro(%rip), %eax
        movslq  %eax, %rdx
        movq    SOMMA(%rip), %rax
        addq    %rdx, %rax
        movq    %rax, SOMMA(%rip)
```

Codice macchina

```
f30f 1efa 4883 ec08 488b 05d9 2f00 0048
85c0 7402 ffd0 4883 c408 c300 0000 0000
ff35 922f 0000 f2ff 2593 2f00 000f 1f00
f30f 1efa 6800 0000 00f2 e9e1 ffff ff90
f30f 1efa 6801 0000 00f2 e9d1 ffff ff90
f30f 1efa f2ff 259d 2f00 000f 1f44 0000
f30f 1efa f2ff 255d 2f00 000f 1f44 0000
f30f 1efa f2ff 2555 2f00 000f 1f44 0000
f30f 1efa 31ed 4989 d15e 4889 e248 83e4
f050 544c 8d05 c601 0000 488d 0d4f 0100
0048 8d3d c100 0000 ff15 322f 0000 f490
488d 3d61 2f00 0048 8d05 5a2f 0000 4839
f874 1548 8b05 0e2f 0000 4885 c074 09ff
e00f 1f80 0000 0000 c30f 1f80 0000 0000
488d 3d31 2f00 0048 8d35 2a2f 0000 4829
```

Si veda per esempio
[BK, capitoli 2 e 4.1]

Dal codice macchina al processo

- Abbiamo già accennato che il termine “**programma**” può riferirsi a tre cose diverse, le prime due le abbiamo appena viste:
  - Il **file sorgente, o codice sorgente**, cioè un testo normalmente prodotto da una persona, e normalmente scritto in un file di testo. Leggibile da un essere umano ma non eseguibile da una macchina;
  - Il **file eseguibile**, prodotto dal compilatore a partire dai file sorgenti. Contiene istruzioni da eseguire in un calcolatore. Leggibile da una macchina ma non da un essere umano;

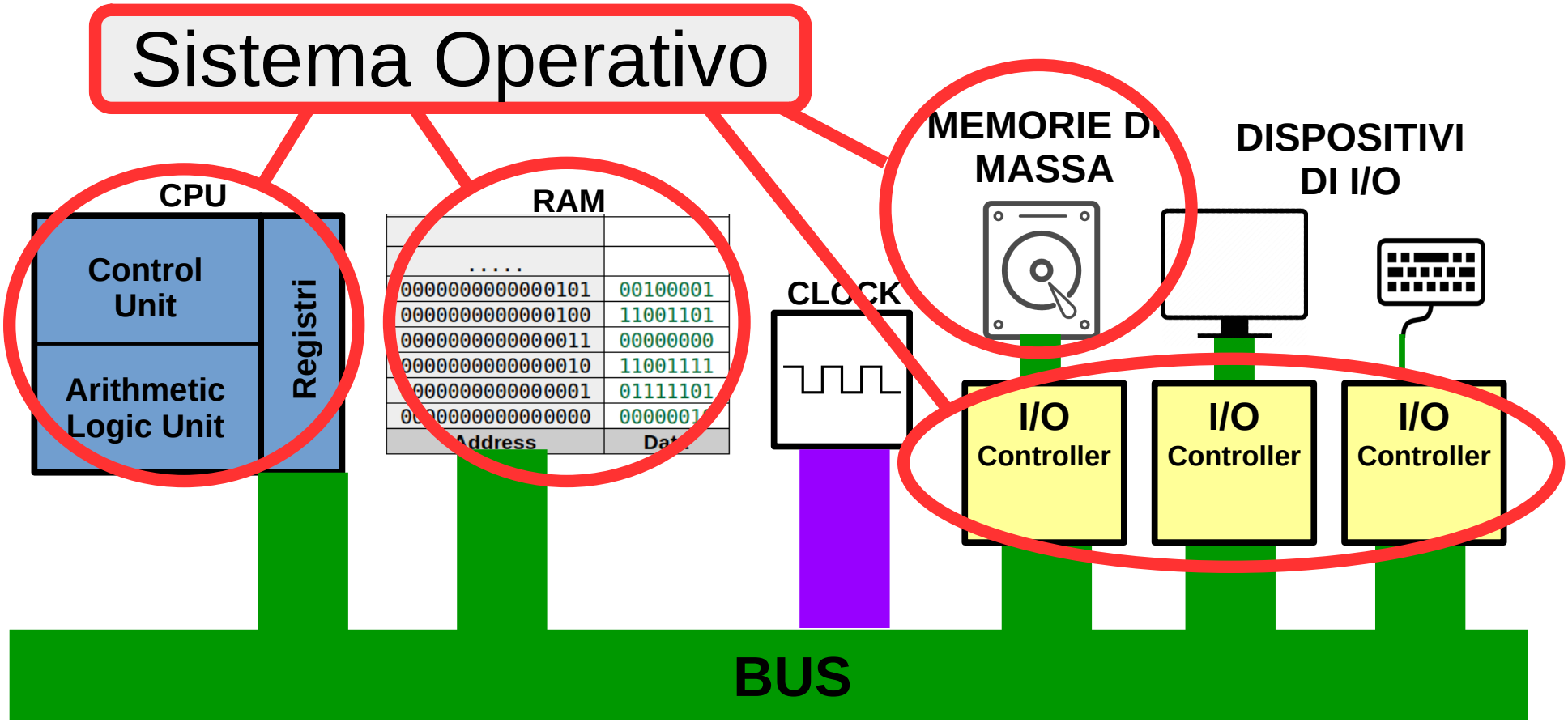
- Il file eseguibile non è ancora eseguito, finché qualcuno o qualcosa non esegue. Un programma che si trova in esecuzione in una macchina si dice anche **processo**. Ma come accade che un file eseguibile venga effettivamente eseguito?

Cosa succede quando viene eseguito un programma?

- La CPU esegue un ciclo continuo che comprende tre operazioni:
  - **Fetch (Recupero):** Ottiene l'istruzione da eseguire dalla memoria
  - **Decode (Decodifica):** Interpreta che azioni l'istruzione dice di fare
  - **Execute (Esecuzione):** Esegue effettivamente l'istruzione
- Quando un programma viene caricato dalla memoria di massa in memoria, cioè quando diventa un processo in esecuzione, nel calcolatore avvengono le seguenti operazioni:
  - La CPU viene informata della prima istruzione del programma tramite un contatore detto program counter;
  - Il programma viene eseguito seguendo le istruzioni che sono caricate in memoria;
  - Il processore esegue le istruzioni leggendole dalla memoria e utilizza la stessa memoria per conservare dati e variabili del programma

Sistema operativo

- C'è un **programma** particolare che permette al computer di funzionare;
- È il programma che viene caricato per primo nella memoria del computer, rimane sempre in esecuzione, e svolge alcune mansioni fondamentali;
- Questo programma si chiama **Sistema Operativo** (Operating System, spesso abbreviato in OS);
- Ne conoscete probabilmente alcune famiglie, per esempio Windows, MacOS, Linux, Android, iOS. Ne esistono molti altri.

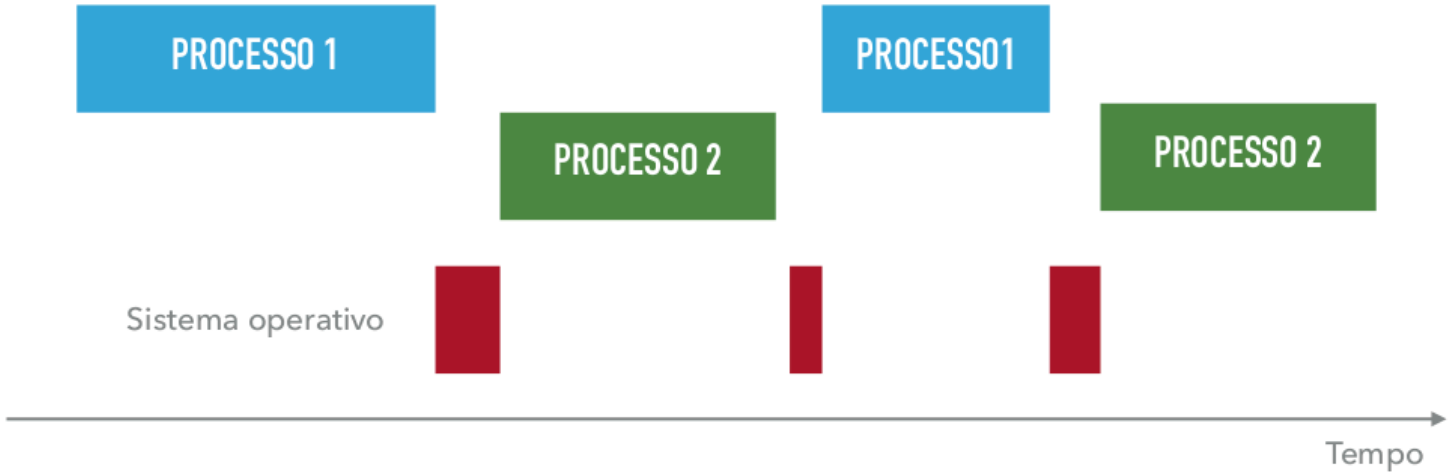

Sistema Operativo
CPU
Control Unit
Arithmetic Logic Unit
Registri
RAM
0000000000000101 00100001
0000000000000100 11001101
0000000000000011 00000000
0000000000000010 11001111
0000000000000001 01111101
Address
CLOCK
MEMORIE DI MASSA
DISPOSITIVI DI I/O
I/O Controller
I/O Controller
I/O Controller
BUS



Questi sono i <u>compiti principali di un sistema operativo</u>

Nota: Come vedete non viene citata l'interfaccia con l'utente.

Scheduler e gestione dei processi



Uno *scheduler* del sistema operativo si occupa di distribuire il tempo di elaborazione della CPU attraverso diversi programmai (incluso il SO stesso), dando l'impressione che più processi siano in esecuzione simultaneamente.

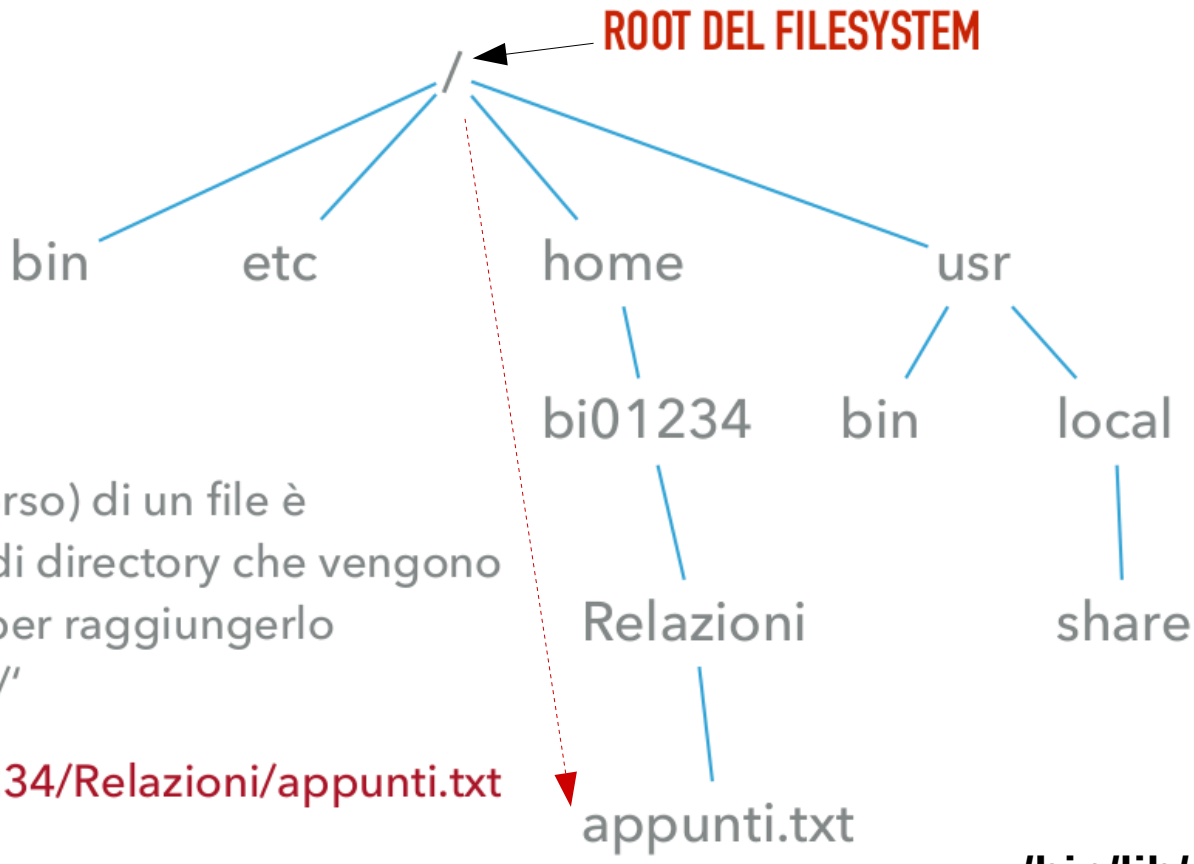
Gestione della memoria di lavoro e delle operazioni di I/O

- Quando un processo vuole accedere a una *risorsa,* la deve richiedere al SO;
- Anche quando al programma serve della memoria, la deve richiedere al sistema operativo;
- Il sistema operativo tiene la memoria di processi diversi separata;
- Lo stesso per l'I/O: è il sistema operativo che interagisce direttamente con l'hardware e media tutti gli accessi;

Gestione del file system

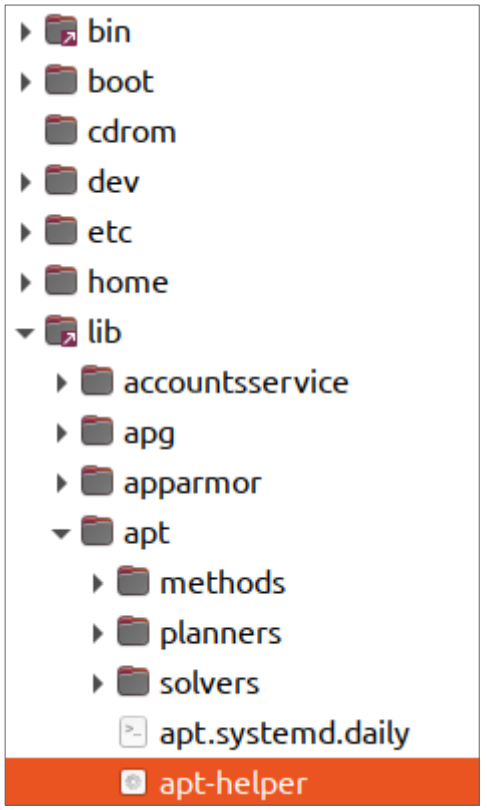
- Generalmente non vogliamo che i programmi vedano direttamente la memoria di massa;
- Il sistema operativo crea una astrazione (il **file**) che viene visto come usa sequenza di byte a cui è dato un nome;
- Il sistema operativo permette di organizzare i file in un file system, cioè una struttura gerarchica tramite le **directory**, che a loro volta possono contenere file o altre directory.
- **Nota terminologica**: “Directory”, “folder”, “cartella” sono sinonimi

Struttura gerarchica del file system



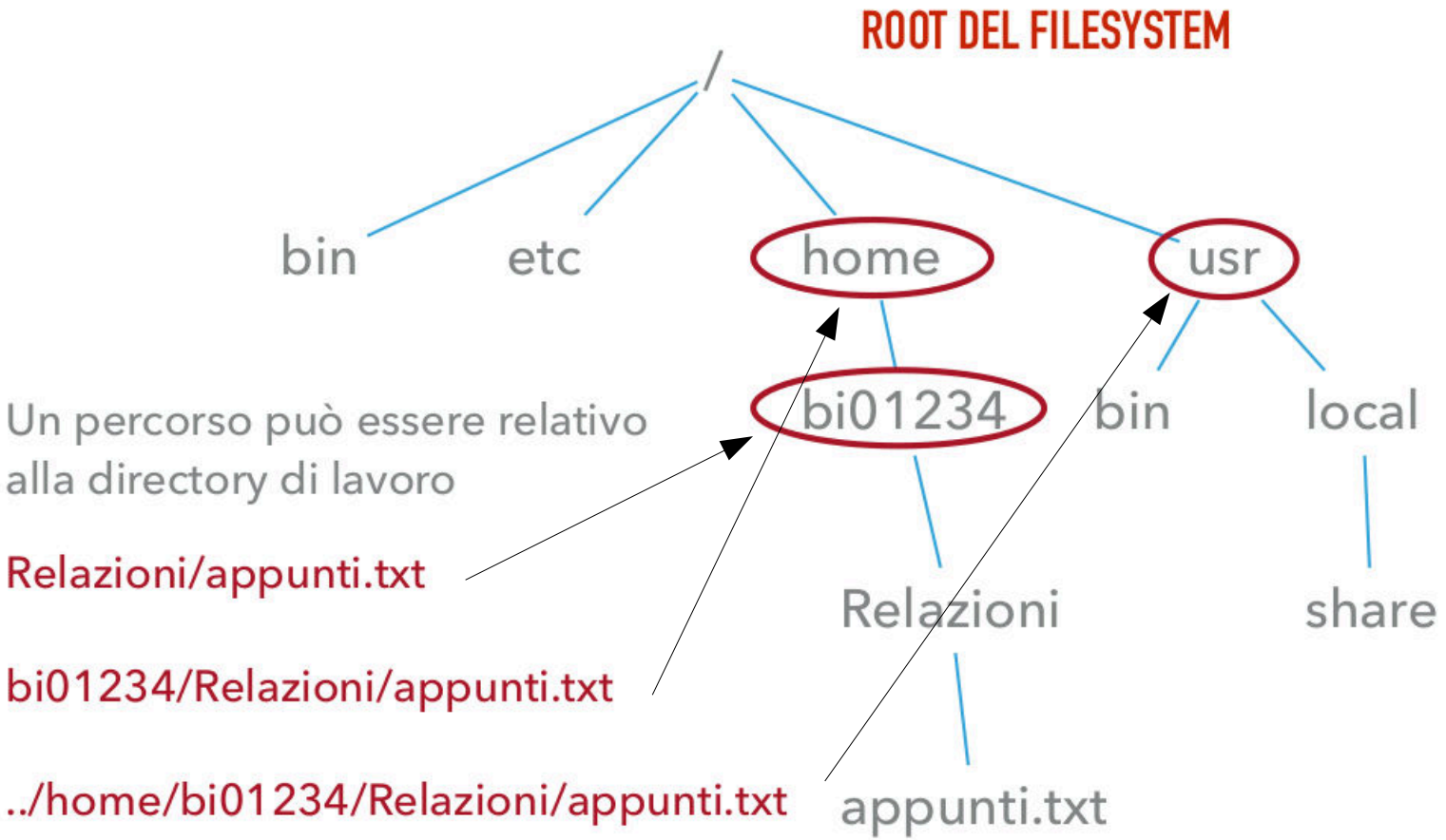
Il path (percorso) di un file è
la sequenza di directory che vengono
attraversate per raggiungerlo
separate da '/'

/home/bi01234/Relazioni/appunti.txt



/bin/lib/apt/solvers/apt-helper

Percorso (o path) assoluto e relativo



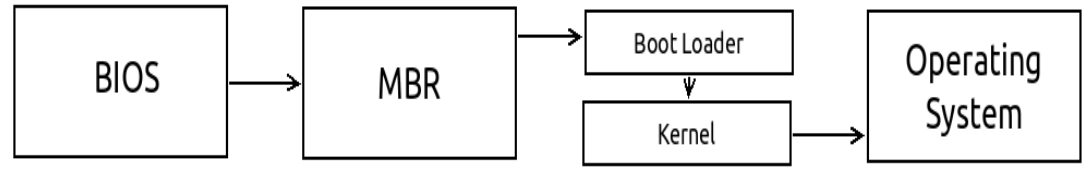
Una parentesi sull'avvio (boot) di un computer

- Quando il computer è spento, tutte le informazioni sono salvate sulle memorie di massa, incluso il sistema operativo
- Quando si accende, qualcosa deve prelevare il sistema operativo dal disco fisso (o dalla chiavetta USB, o da altri dispositivi, incluse le schede di rete) e caricarlo in memoria
- La fase che va dall'accensione del computer al momento in cui il sistema operativo è completamente caricato in memoria ed è pronto a eseguire programmi si chiama **boot.**
- **Chi è che va a prendere il sistema operativo dal disco?**

Firmware

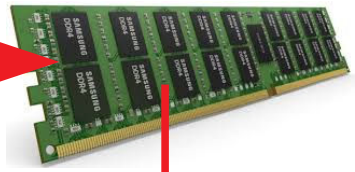
- È un software che svolge specifiche istruzioni codificato in modo non volatile all'interno di un circuito elettronico (eventualmente riprogrammabile)
- Per questo a volte si dice che è a metà tra software e hardware
- Si trovano firmware un po' ovunque: lavatrici, televisori, automobili, cellulari, computer, periferiche, eccetera…
- Nei computer esiste un particolare firmare che si occupa del suo avvio, spesso per brevità chiamato BIOS

L'avvio di un computer: il **boot**

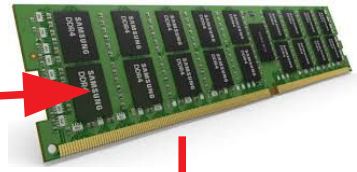


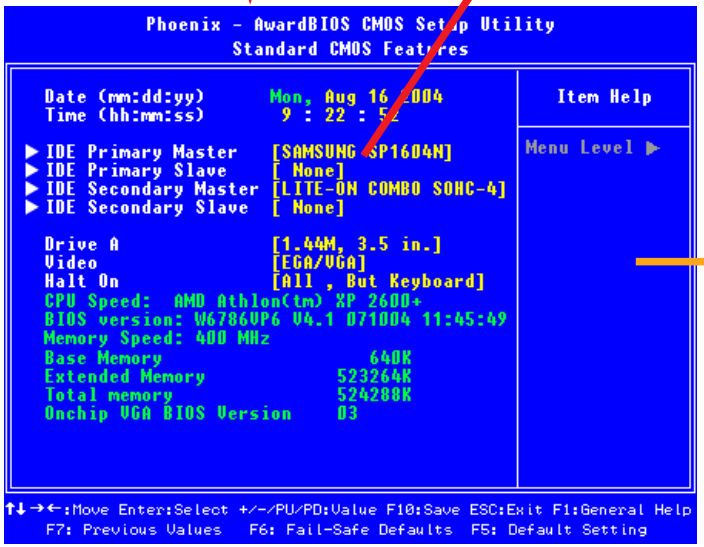


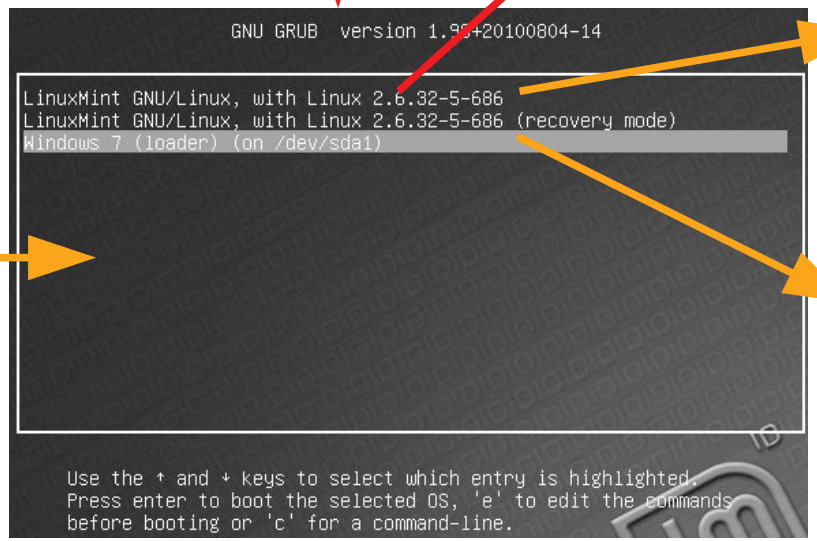


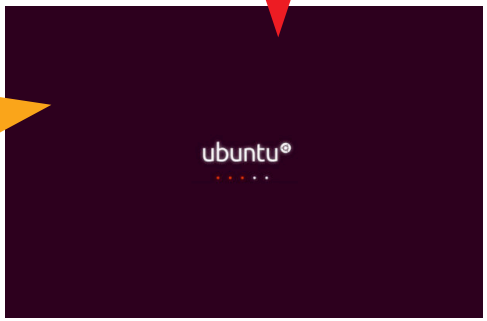














BIOS

BOOT LOADER

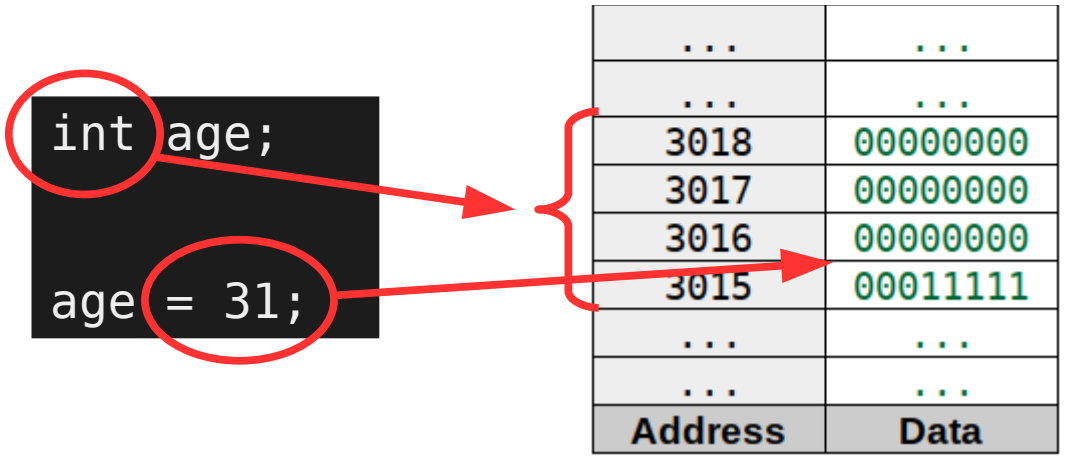
SISTEMA OPERATIVO

Nota: il BIOS oggi è sempre più spesso rimpiazzato da UEFI nei PC. Altri tipo di computer possono avere tecnologie diverse e varianti di funzionamento, ma la sostanza non si discosta troppo

Codifica, Rappresentazione e Organizzazione dei dati

Informazioni	Rappresentazione astratta dei dati	Rappresentazione logica e fisica dei dati
La mia età	Numero intero	Sequenza di bit in memoria

Nota: Da qui in poi, salvo diversamente specificato, gli esempi e le nozioni sul codice si riferiranno al linguaggio C.



Dati, variabili, tipi

- Gli oggetti su cui opera un programma sono le variabili;
- Una variabile ha un nome, un tipo, un valore; occupa spazio nella memoria di lavoro (RAM);
  - **Nome** (identificatore): `a, d, x, somma, average, x1`
  - **Tipo**: “numero intero”, “numero reale”, carattere, sequenza di numeri, . . .
- Di norma, lo spazio occupato dipende dal tipo;
- Ogni tipo corrisponde a uno spazio fissato a priori (quindi . . .);
- Il tipo determina i valori che la variabile può avere e le operazioni che si possono eseguire.

Usare le variabili in C

- Nel C prima di usare una variabile si deve dichiararne il tipo, in questo modo:

```
tipo identificatore;
```

- Tipi primitivi in C:
  - Interi (integer): `short, int, long`.
  - Virgola mobile (floating-point): `float, double`.
  - Caratteri: `char`

```
int n, nelementi, i;
float x, z, height, largh;
char sigla, id;
```

Assegnamento

- La forma generale dell'assegnamento è:

```
identificatore = espressione;
```

- Per assegnare un valore a una variabile si usa il simbolo “=”
- Quindi “=” non significa... uguale
- Ha un “verso”, cioè si assegna al nome a sinistra il valore che sta a destra
- l’espressione viene valutata con i valori correnti delle variabili; il risultato viene scritto nella cella di memoria occupata dalla variabile nominata a sinistra

Ha senso → `x = x + 1;`

Non ha senso → `x + y = 2;`

Esempi

```
int base, altezza, area;

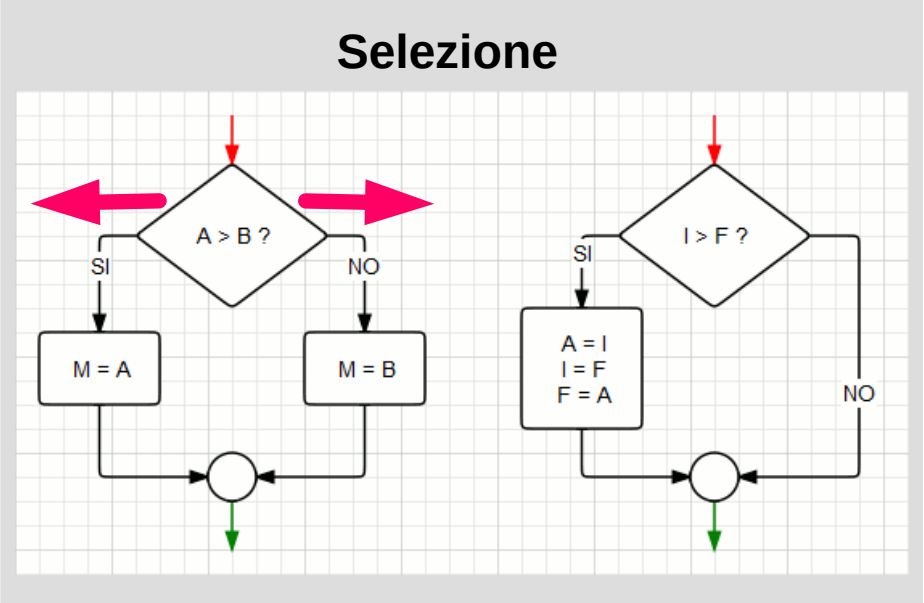
base = 9;
altezza = 3;

area = (base * altezza)/2;
```

Attenzione: questo codice restituisce come risultato 13!

Selezione in C

- Se la condizione è valida, allora:
  - esegui queste istruzioni;
- altrimenti:
  - esegui queste altre istruzioni



- La condizione può essere VERA (1) o FALSA (0)
- Condizione = espressione booleana
- Esempi:

```
x<y
n >= 0
denom == 0
denom != 0
```

Selezione in C

```
if ( condizione ) {
    blocco di istruzioni
}

if ( condizione ) {
    blocco di istruzioni 1
} else {
    blocco di istruzioni 2
}

if ( condizione1 ) {
    blocco di istruzioni 1
} else if ( condizione2 ) {
    blocco di istruzioni 2
} else {
    blocco di istruzioni 3
}
```

Esempio:

```
int a, b, max, min;

...

if ( a <= b ) {
    max = b; min = a;
}
else {
    max = a; min = b;
}

...
```

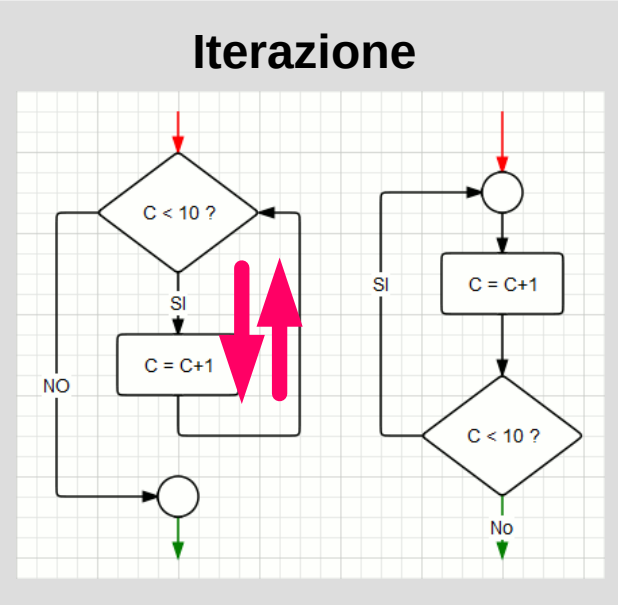
Iterazione in C

- Fintantoché (while) vale c, ripeti queste istruzioni

- Come per la selezione, un blocco di istruzioni dev'essere racchiuso fra parentesi graffe

```
while ( condizione ) {
    blocco di istruzioni
}

do {
    blocco di istruzioni
} while ( condizione )
```



Iterazione in C

```
while ( condizione ) {
    blocco di istruzioni
}

do {
    blocco di istruzioni
} while ( condizione )
```

Esempio

```
int euro;
float lire, cambio;
cambio = 1936.27;
euro = 4;

while ( euro < 10000 ) {
    lire = euro * cambio;
    euro = euro * 2;
    /* Stampa il valore di euro e di lire */
    printf("%d\n", euro);
}
```

Correttezza, finitezza e costo di un algoritmo

- Prendiamo un caso classico di problema risolvibile per via algoritmica, quello dell'ordinamento
- Dovremo dimostrare o almeno fare attenzione al fatto che la soluzione algoritmica proposta sia
  - Corretta
  - Finita
- Dovremo anche chiederci quale sia il costo computazionale dell'algoritmo, come indicarlo e come confrontare due algoritmi che risolvono lo stesso problema attraverso il loro costo.

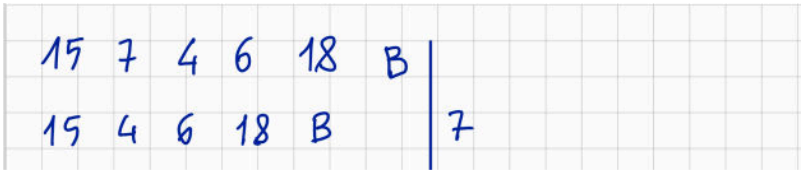
Algoritmo di ordinamento



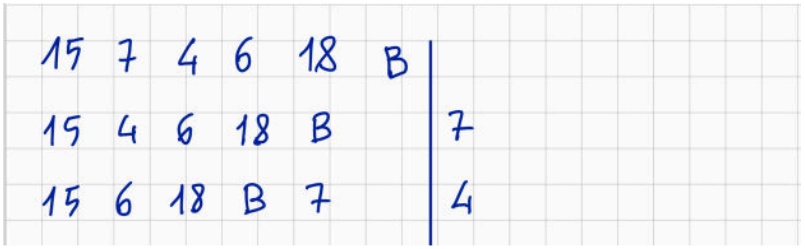
Nei seguenti esempi immaginiamo dei mazzi di carte numerate, su cui sono scritti numeri interi

Una carta non ha alcun numero, è semplicemente un cartoncino blu

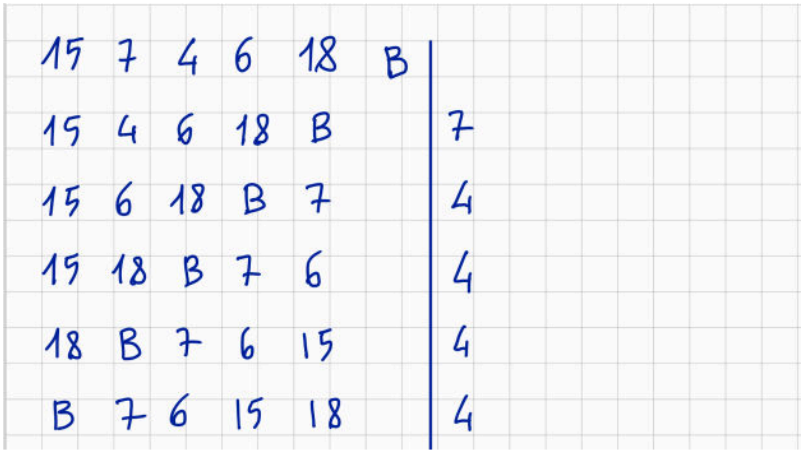
Algoritmo di ordinamento



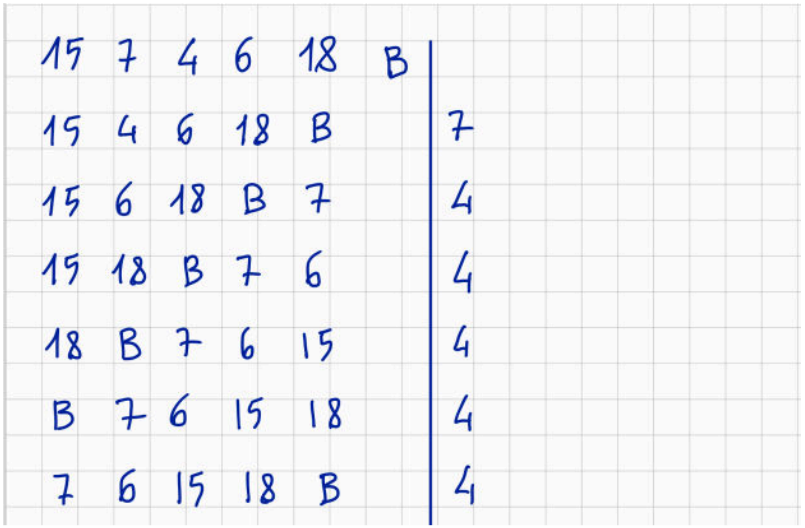
Algoritmo di ordinamento



Algoritmo di ordinamento



Algoritmo di ordinamento



Algoritmo di ordinamento



Analisi del costo computazionale

- Per un mazzo di N carte, nel primo ciclo, fino a ritrovare la carta blu del mazzo, ho eseguito nel peggiore dei casi N-1 confronti
- Nel secondo ciclo, (N-1) - 1 = N-2
- Nel peggiore dei casi ho eseguito un numero di confronti pari a

$$\sum_{1}^{N-1} i = \frac{N*(N-1)}{2}$$

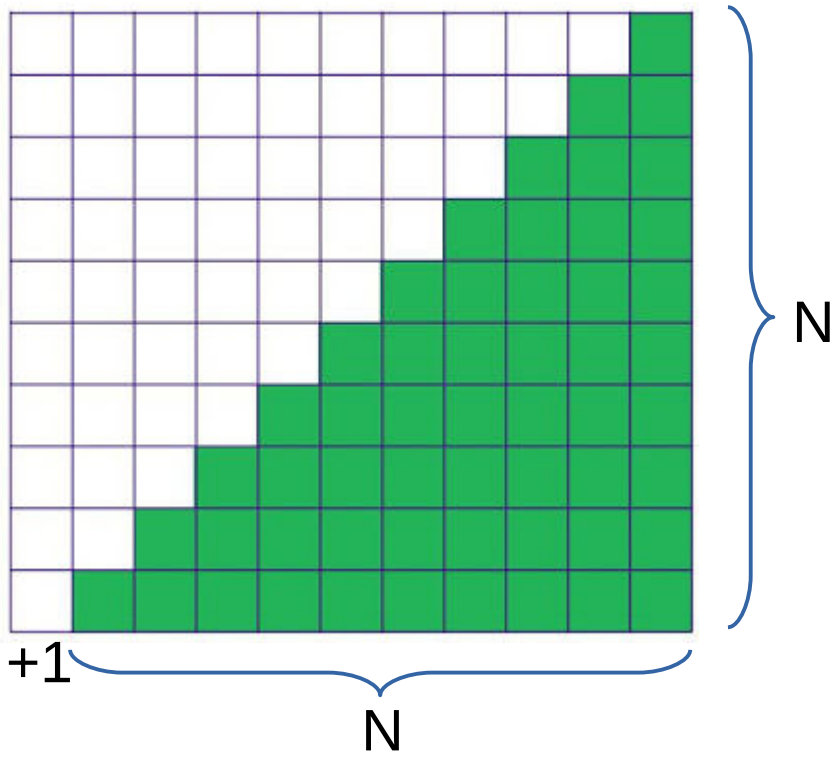
- Se avessi per esempio 100 carte, avrei eseguito 4950 confronti
- La cosa importante da notare è il termine che presenta N con esponente più grande, in questo caso N^2
- Si dice che il costo computazionale di questo algoritmo è $O(N^2)$



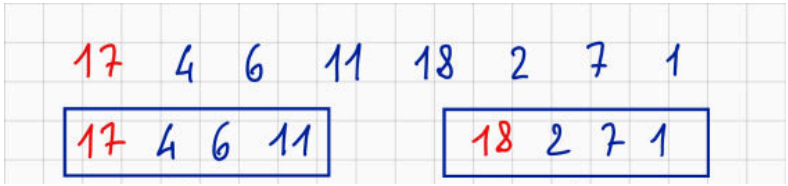
Bonus slide: Somma dei primi N numeri

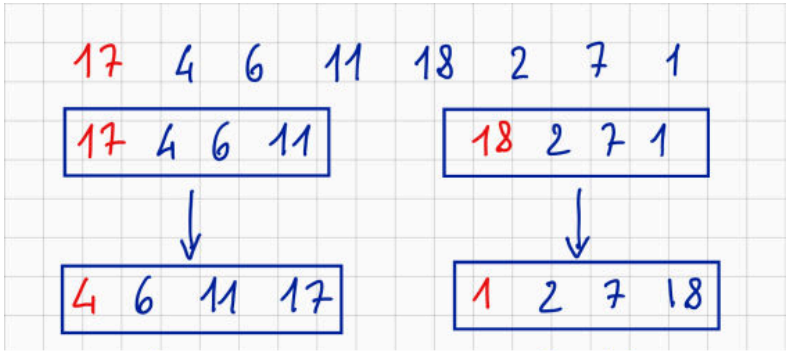
- L'area di questo rettangolo è N * N + 1
- La regione verde e la regione bianca sono uguali
- La regione verde corrisponde alla somma dei primi N numeri
- La somma dei primi N numeri è pari alla metà dell'area del rettangolo, quindi:

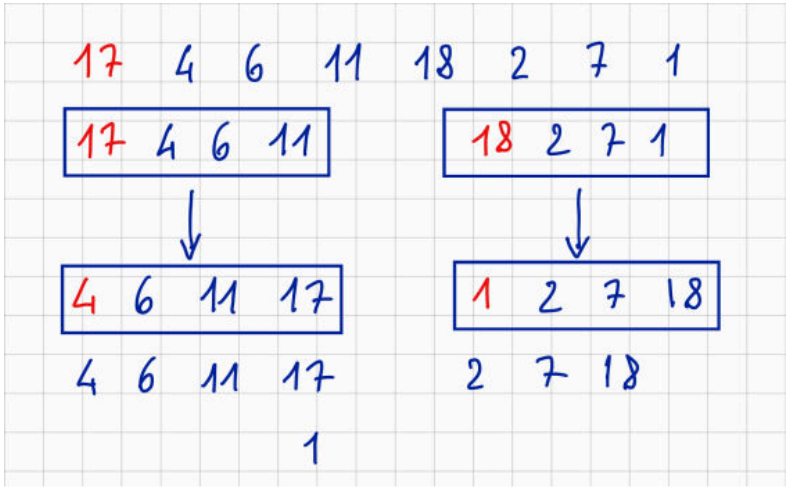
$$\sum_{1}^{N} i = \frac{N * (N+1)}{2}$$

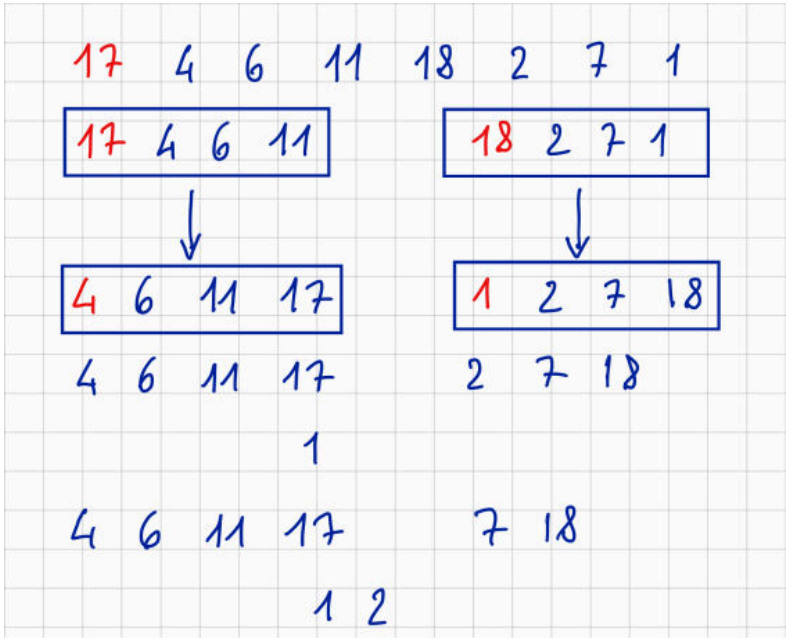


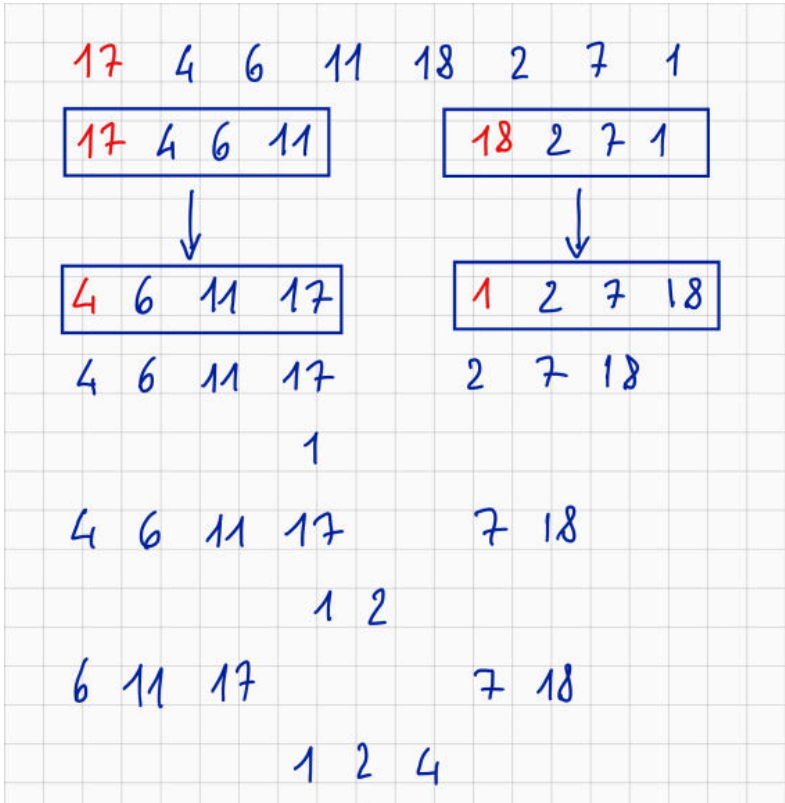

17 4 6 11 18 2 7 1

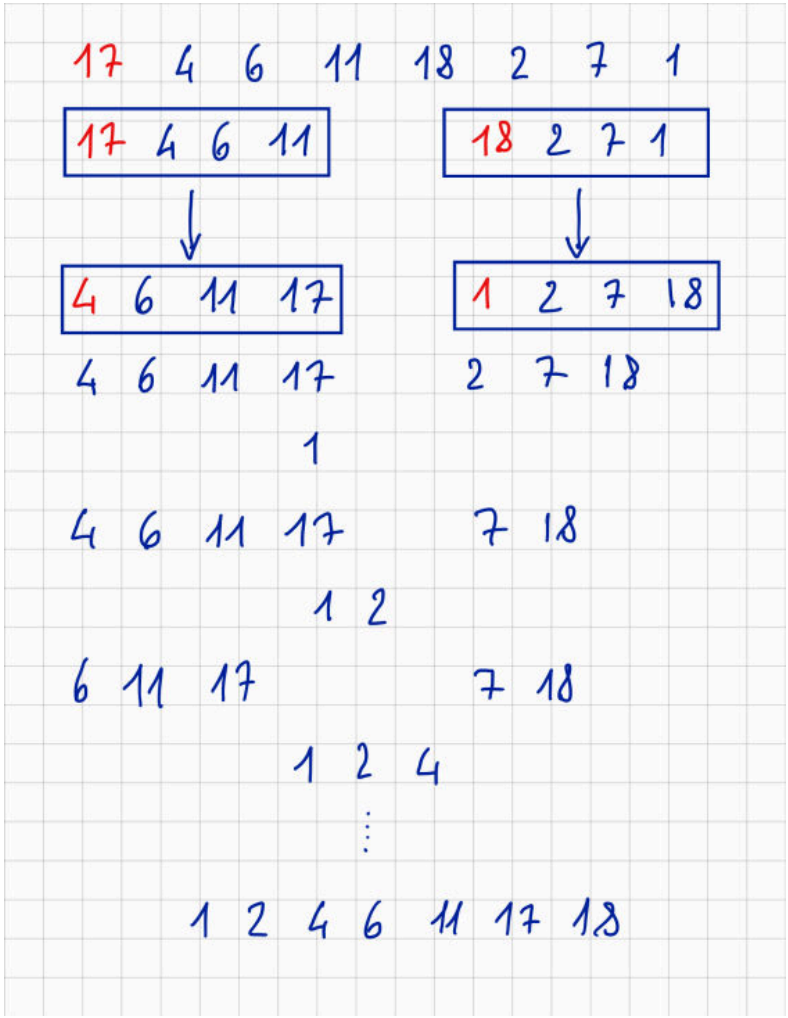

17 4 6 11 18 2 7 1
17 4 6 11
18 2 7 1


17 4 6 11 18 2 7 1
17 4 6 11
18 2 7 1
4 6 11 17
1 2 7 18


17 4 6 11 18 2 7 1
17 4 6 11
18 2 7 1
4 6 11 17
1 2 7 18
4 6 11 17
2 7 18
1


17 4 6 11 18 2 7 1
17 4 6 11
18 2 7 1
4 6 11 17
1 2 7 18
4 6 11 17
2 7 18
1
4 6 11 17
7 18
1 2


17 4 6 11 18 2 7 1
17 4 6 11
18 2 7 1
4 6 11 17
1 2 7 18
4 6 11 17
2 7 18
1
4 6 11 17
7 18
1 2
6 11 17
7 18
1 2 4

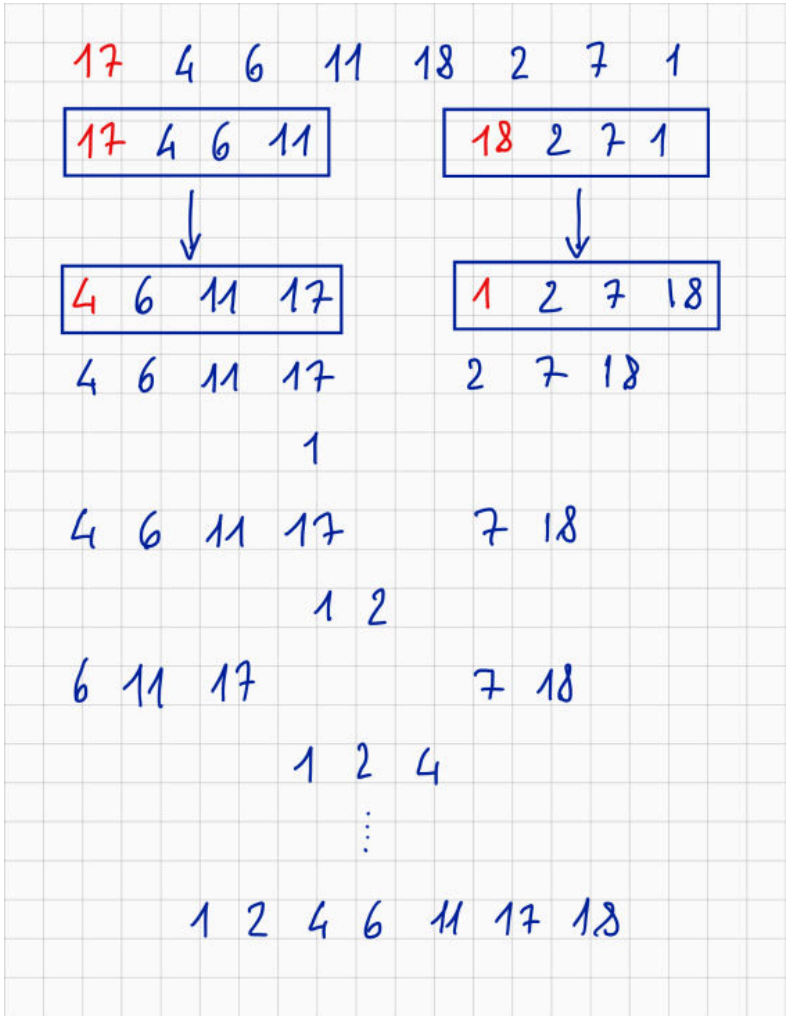

17 4 6 11 18 2 7 1
17 4 6 11
18 2 7 1
4 6 11 17
1 2 7 18
4 6 11 17
2 7 18
1
4 6 11 17
7 18
1 2
6 11 17
7 18
1 2 4
1 2 4 6 11 17 18

Analisi del costo computazionale

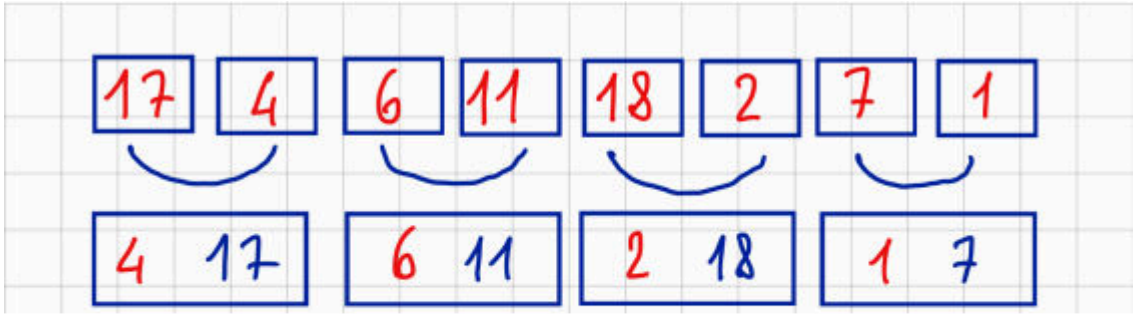
- Per ordinare entrambi i sotto mazzi di N/2 carte, nel peggiore dei casi ho eseguito un numero di confronti pari a

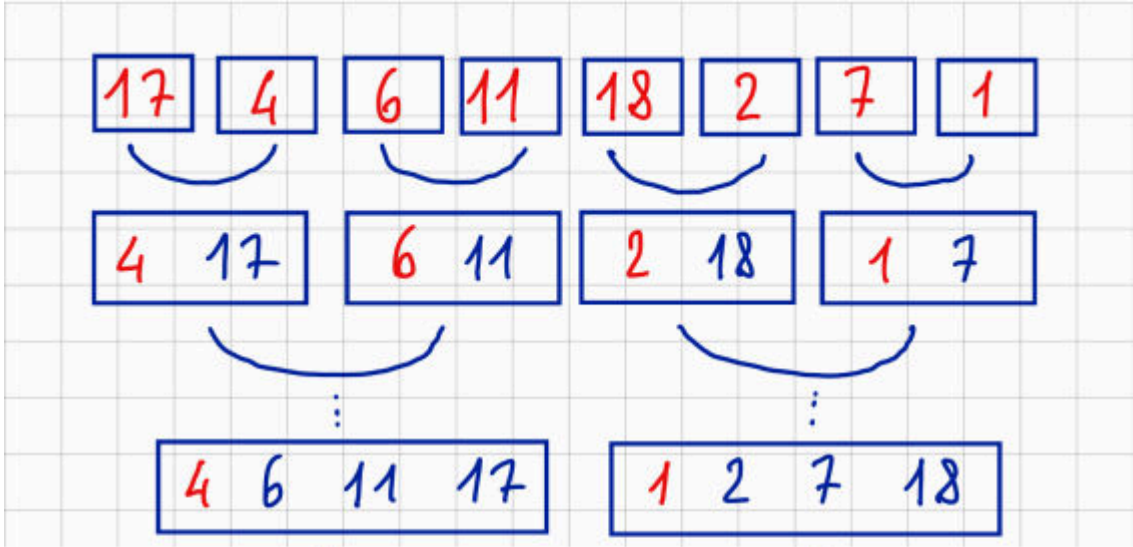
$$2 * \sum_{1}^{N/2-1} i = \frac{N/2 * (N/2-1)}{2} = 2450$$

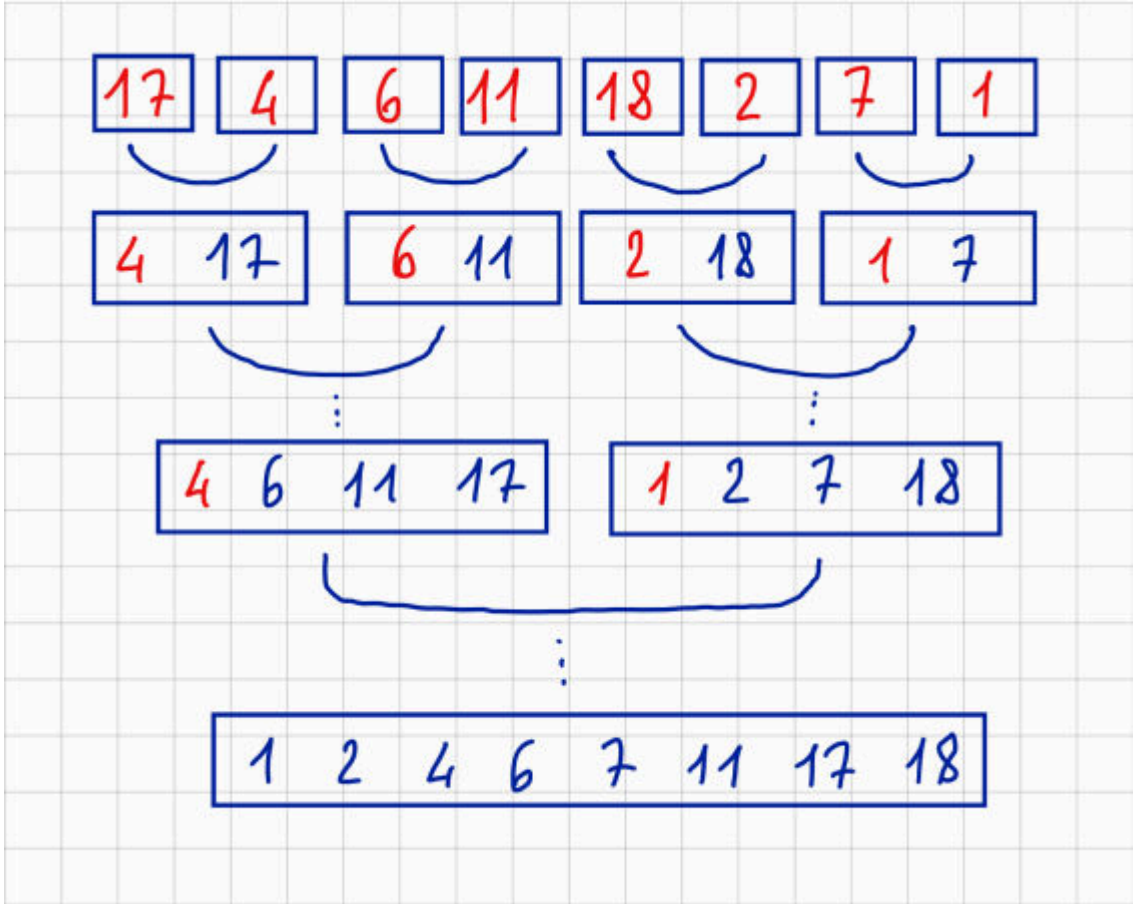
- Notate che $2 * (N/2)^2 < N^2$
- Per effettuare la fusione dei due sottomazzi ordinati, nel peggiore dei casi ho effettuato N-1 confronti.
- In totale, ho effettuato 2450 + 99 = 2549 confronti, che è circa la metà del caso precedente, in cui erano richiesti per 100 carte 4950 confronti
- Probabilmente questo algoritmo ha un costo computazionale inferiore.




17
4
6
11
18
2
7
1


17 4 6 11 18 2 7 1
4 17 6 11 2 18 1 7


17 4 6 11 18 2 7 1
4 17 6 11 2 18 1 7
4 6 11 17
1 2 7 18


17 4 6 11 18 2 7 1
4 17 6 11 2 18 1 7
4 6 11 17 1 2 7 18
1 2 4 6 7 11 17 18

Analisi del costo computazionale

Carte per mazzo	N. mazzi	Confronti per mazzo			
1	32				
		1	16	$2^1 - 1$	2^4
2	16				
		3	24	$2^2 - 1$	2^3
4	8				
		7	28	$2^3 - 1$	2^2
8	4				
		15	30	$2^4 - 1$	2^1
16	2				
		31	31	$2^5 - 1$	2^0
32	1				

*

Il costo di questo algoritmo è O(N * $\log_2$(N))

Vedere le prime pagine dispense sul sito (lif_dispensa.pdf)

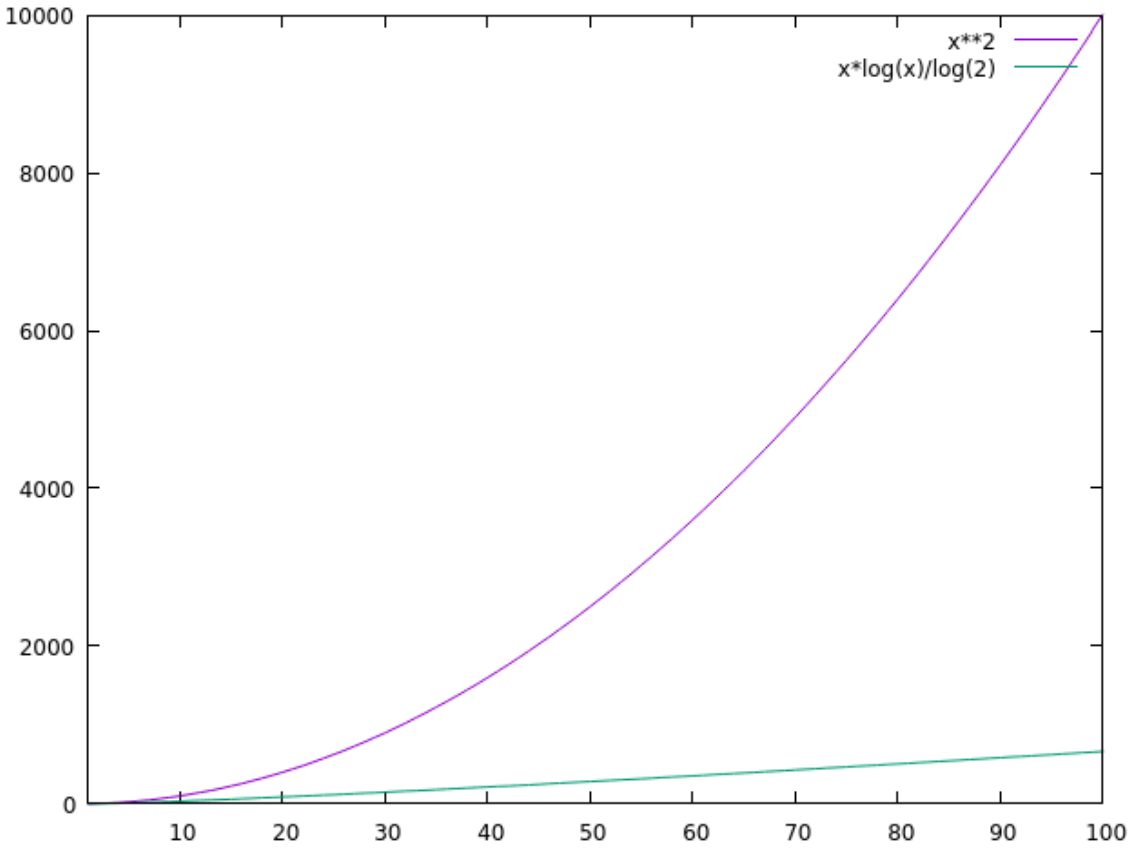
- Supponiamo di avere 32 numeri (o carte) da ordinare
- Le prime due colonne mostrano l'inizio di ogni fase di fusione;
- La prima riga fotografa la situazione nella quale abbiamo 32 mazzi di 1 carta ciascuno;
- Le righe intermedie mostrano il numero di confronti necessari per ogni fusione e il numero di fusioni da compiere.
- Ad esempio, la seconda riga dice che si devono compiere 16 operazioni di fusione, ognuna delle quali richiede 1 confronto per fondere 2 mazzi di 1 carta ciascuno. Il risultato sarà un gruppo di 16 mazzi di due carte ciascuno, come indicato nella terza riga.
- Il numero totale di confronti per il passaggio da una fase alla successiva è dato dal prodotto del numero nella seconda colonna per quello della terza.
- Riscritti in forma diversa, il numero di confronti per mazzo e il numero di mazzi che si ottengono compaiono nelle colonne quinta e sesta.
- Se trascuriamo il '-1', otteniamo come risultato 2^5 per ogni riga: notare che questo numero è proprio il numero totale di cartoncini, cioè $2^5 = 32 = N$.
- Quante sono le fasi distinte di fusione? Se partiamo con N carte, e a ogni passo dimezziamo la taglia dei mazzetti. Per arrivare a mazzetti di un cartoncino occorrono $\log_2$ (N) passaggi. Quindi, procedendo a ritroso occorrono altrettante fasi per ricostruire il mazzo intero.
- Quindi nel peggiore dei casi questo algoritmo richiede

  O(N * $\log_2$(N)) confronti.

Andamento di N^2 vs $N * \log_2(N)$

Ricordando che $\log_2(N) = \ln(N) / \ln(2)$

```
gnuplot> set xrange [1:100]
gnuplot> plot x**2
gnuplot> replot x*log(x)/log(2)
gnuplot>
```



Per le esercitazioni

- Controllare il materiale di studio e il materiale di supporto tecnico sul sito del corso;
- Dotarsi di un terminale (o shell testuale) di tipo Unix (Linux, MacOs, o le varie soluzioni per Windows)
- Dotarsi di gcc
- Dotarsi di un editor di testo, o *text editor* (Vim, Gedit, Notepad++, Atom,...)
- Studiare le Note sull'interazione fra utente e sistema operativo nella sezione Esercitazioni del sito del corso;
- Leggere il file vigcc.pdf nella cartella Materiale per la prima esercitazione;
- Vi ricordo che la prima parte sarà una spiegazione, verrà registrata, non perdetela.